



Debre Berhan University

College of Computing

Department of Computer Science

System Development Module

Part I: Software Engineering

Part II: Web Programming

 **Part III: Database Systems**

March 2023
Debre Berhan,
Ethiopia

Contents for fundamental of Database

Chapter One	5
Introduction to Database Systems.....	5
1.1 Introduction.....	5
1.2 Database System versus File System.....	5
1.3 Characteristics of the Database Approach	5
1.4 Actors on the scene	6
Chapter Two – Database System Architecture	7
2.1 Data models	7
Types of Data Models.....	7
2.2 Schemas and Instances.....	7
2.3 Three-schema Architecture and Data Independence.....	7
2.4 Data Independence.....	8
Chapter Three – Database Modeling	9
3.1 Database Modeling	9
3.2 Phases of database design	9
Requirement Analysis.....	9
Conceptual Database Design	9
Logical database design (Data model mapping)	9
Physical Design.....	9
3.3 ERD (Entity Relationship Diagram)	9
Entity and Attribute.....	10
Types of Attributes	10
3.4 Mapping ER-models to relational tables.....	12
3.5 Enhanced Entity Relationship (EER) Model	12
3.6 The Relational Database Model.....	13
3.7 Relational Model Concepts in DBMS	13
3.8 Relational Constraints	14
Chapter Four	15
Functional Dependency and Normalization.....	15
4.1 Functional Dependency.....	15
4.2 Types of Functional Dependency	15
4.3 Database Anomalies.....	16

4.4	Normalization	16
	Normal forms	16
Chapter Five.....		19
5.1	Introduction.....	19
5.2	Choosing Attribute Data Types.....	19
5.3	Data/File Storage: Categories	20
5.4	File Organizations	20
5.5	Index	21
	Types of Indexes	21
Chapter Six.....		24
The Relational Algebra and Relational Calculus		24
6.1	Relational algebra	24
6.2	Relational Calculus	24
	Tuple Relational Calculus.....	25
Chapter Seven - SQL		26
7.1	SQL	26
7.1.1	DDL (data definition language)	26
7.1.2	DML (data manipulation language).....	26

Contents of Advanced database

Chapter 1: Query Processing and Optimization.....		27
1.1	Query Processing and Optimization: Why?	27
1.2	Query Processing Steps.....	29
1.3	Query Decomposition	29
1.4	Query Optimization	30
1.5	Pipelining	44
Chapter 2: Database Security and Authorization		47
2.1	Data Integrity	47
2.2	Database Security.....	52
2.2.1	DBMS Security Support	53
2.2.2	Permissions and Privilege	53
2.2.3	Authorization Rules	54
2.2.4	Privileges in SQL	54

2.3	Backup and Recovery	60
Chapter 3: Transaction Processing Concepts		63
3.1	Introduction to Transaction	63
3.2	Transaction and System Concepts	64
3.3	Transaction Processing	67
3.4	Concept of Schedules and Serializability	70
3.5	Transaction Support in SQL	76
Chapter 4: Concurrency Controlling Techniques		79
4.1	Database Concurrency Control	79
4.2	Concurrency Control Techniques	81
4.2.1	Locking	82
4.2.2	Two-Phase Locking Techniques: The algorithm	86
4.2.3	Timestamp based concurrency control algorithm	90
4.2.4	Multiversion Concurrency Control Techniques	92
4.2.5	Validation (Optimistic) Concurrency Control Schemes	94
4.2.6	Multiple Granularity Locking	95
Chapter 5: Database Recovery Techniques		97
5.1	Database Recovery	97
5.2	Transaction and Recovery	97
Chapter 6: Distributed Databases and Client-Server Architectures		113
6.1	Distributed Database Concepts	113
6.2	Data Replication and Fragmentation: Distributed data storage	116
6.3	Types of Distributed Database Systems	117
6.4	Query Processing in Distributed Databases	119
6.5	Concurrency Control and Recovery	121
6.5.1	Distributed Concurrency control	122
6.6	Client-Server Database Architecture	124

Chapter One

Introduction to Database Systems

1.1 Introduction

Data is known facts that can be recorded and that have implicit meaning. Database is a collection of related data. For example, consider the names, telephone numbers, and addresses of the people you know. Nowadays, this data is typically stored in mobile phones, which have their own simple database software. This collection of related data with an implicit meaning is a **database**. A database can be defined as a self-describing collection of integrated records.

A database can be of any size and complexity. A **personal database** is designed for use by a single person on a single computer. Such a database usually has a rather simple structure and a relatively small size. An **enterprise database** can be huge. Enterprise databases may model the information flow of entire large organizations.

Database Management System (DBMS) is then a tool for creating and managing databases. DBMS is a software that facilitates the processes of defining, constructing, manipulating, and sharing databases. Example: MySQL server, MS ACCESS, Microsoft SQL server.

1.2 Database System versus File System

The traditional file processing system is file-directory structure supported by a conventional operating system. A file system organization of data lacks a number of major features of a database system, such as Data redundancy and inconsistency, Difficulty in accessing data, Data isolation, Integrity problems, Atomicity problems, Concurrent access anomalies.

1.3 Characteristics of the Database Approach

The main characteristics of the database approach are Self-describing nature, Data abstraction and Insulation between programs and data, Support of multiple views of the data, sharing of data and multiuser transaction processing.

Self-describing nature - A fundamental characteristic of the database approach is that the database system contains not only the database itself (data) but also a complete definition or description of the database structure and constraints. This information is called meta-data, and it describes the structure of the database.

Data abstraction and Insulation between programs and data- Data Abstraction is a process of hiding unwanted or irrelevant details from the end user. It provides a different view and helps in achieving data independence which is used to enhance the security of data.

Support of multiple views of the data - A view may be a subset of the database or it may contain virtual data that is derived from the database files but is not explicitly stored.

Sharing of data and multiuser transaction processing - A multiuser DBMS must allow multiple users to access the database at the same time. This is essential if data for multiple applications is to be integrated and maintained in a single database.

1.4 Actors on the scene

The people whose jobs involve the day-to-day use of a large database are called **actors on the scene**. In a database environment, the primary resource is the database itself, and the secondary resource is the DBMS and related software. Administering these resources is the responsibility of the **database administrator (DBA)**. The DBA is responsible for authorizing access to the database, coordinating and monitoring its use, and acquiring software and hardware resources as needed.

Chapter Two – Database System Architecture

2.1 Data models

A data model a collection of concepts that can be used to describe the structure of a database and provides the necessary means to achieve data abstraction. By structure of a database we mean the data types, relationships, and constraints that apply to the data. Most data models also include a set of basic operations for specifying retrievals and updates on the database.

Types of Data Models

1. **High-level or conceptual data models** provide concepts that are close to the way many users perceive data. Entity – Relationship (E/R) model and Object-Oriented Model are examples of conceptual/high level data model.
2. **Low-level or physical data models** provide concepts that describe the details of how data is stored on the computer storage media. Concepts provided by physical data models are generally meant for computer specialists, not for end users.
3. **representational (or implementation) data model** provides concepts that may be easily understood by end users but that are not too far from the way data is organized in computer storage. Representational data models hide many details of data storage on disk but can be implemented on a computer system directly. The widely used relational data models are the **network** and **hierarchical**.

2.2 Schemas and Instances

The description of a database is called the **database schema**, which is specified during database design and is not expected to change frequently. The data in the database at a particular moment of time is called a **database state or snapshot**. It is also called the current set of occurrences or **instances** in the database.

2.3 Three-schema Architecture and Data Independence

The goal of the three-schema architecture is to separate the user applications from the physical database. In this architecture, schemas can be defined at the following three levels:

1. **The internal level has an internal schema**, which describes the physical storage structure of the database. The internal schema uses a physical data model and describes the complete details of data storage and access paths for the database.

2. **The conceptual level has a conceptual schema**, which describes the structure of the whole database for users. The conceptual schema hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints.
3. **The external or view level** includes a number of external schemas or user views. Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group.

2.4 Data Independence

The three-schema architecture can be used to further explain the concept of data independence, which can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level. The two types of data independence:

1. **Logical data independence** is the capacity to change the conceptual schema without having to change external schemas or application programs.
2. **Physical data independence** is the capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas do not need be changed as well.

Chapter Three – Database Modeling

3.1 Database Modeling

High-level conceptual data models provide concepts for presenting data in ways that are close to the way people perceive data. A typical example is the entity relationship model, which uses main concepts like entities, attributes and relationships.

3.2 Phases of database design

The phases of database development life cycle (DDLC) in the Database Management System (DBMS) are explained below.

Requirement Analysis

The most important step in implementing a database system is to find out what is needed i.e what type of a database is required for the business organization, daily volume of data, how much data needs to be stored in the master files etc. In order to collect all this information, a database analyst spends a lot of time within the business organization talking to people, end users and getting acquainted with the day-to-day process.

Conceptual Database Design

In this phase the database designers will make a decision on the database model that perfectly suits the organization's requirement. The database designers will study the documents prepared by the analysis in the requirement analysis stage and then start development of a system model that fulfils the needs.

Logical database design (Data model mapping)

The conceptual design is translated into internal model which includes mapping of all objects i.e design of tables, indexes, views, transaction, access privileges etc.

Physical Design

Physical design or Database implementation needs the formation of special storage related constructs. These constructs consist of storage groups, table spaces, data files, tables etc.

3.3 ERD (Entity Relationship Diagram)

ERD a graphical technique for representing a database schema. It is a modeling tool used to depict graphically a DB design before it is actually implemented. It shows the various entities being modeled and the important relationships among them. ERD has three basic components Entity, Relationship and Attribute.

Entity and Attribute

An **entity** may be an object with a physical existence (for example, a particular person, car, house, or employee) or it may be an object with a conceptual existence (for instance, a company, a job, or a university course). In ER diagrams, an entity is placed in a rectangle.

Each entity has **attributes**, the particular properties that describe it. Entities that do not have key attributes of their own are called **weak entity**. The regular entity types that have a key attribute are called **strong entity types**. Weak entities are identified by being related to specific strong entities in combination with one of their attribute values. A weak entity type has a **partial key**, which is the attribute that can uniquely identify weak entities that are related to the same owner entity.

In ER diagrams, both a weak entity type and its identifying relationship are distinguished by surrounding their boxes and diamonds with double lines. The partial key attribute is underlined with a dashed or dotted line.

Types of Attributes

Composite and Simple (Atomic) Attributes

Composite attributes can be divided into smaller subparts, which represent more basic attributes with independent meanings. Attributes that are not divisible are called simple or atomic attributes.

Multivalued Attributes

Most attributes have a single value for a particular entity; such attributes are called **single-valued**. For example, Age is a single-valued attribute of a person. In some cases an attribute can have a set of values for the same entity. For instance, a College_degrees attribute for a person. Similarly, one person may not have any college degrees, another person may have one, and a third person may have two or more degrees; therefore, different people can have different numbers of values for the College_degrees attribute. Such attributes are called **multivalued**.

Derived Attributes

In some cases, two (or more) attribute values are related. For example, the Age and Birth_date attributes of a person. For a particular person entity, the value of Age can be determined from the current (today's) date and the value of that person's Birth_date. The Age attribute is hence called a **derived attribute** and is said to be derivable from the Birth_date attribute, which is called a stored attribute.

Key Attribute

An important constraint on the entities of an entity type is the key or uniqueness constraint on attributes. An entity type usually has one or more attributes whose values are distinct for each individual entity in the entity set. Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely.

Key

Super Key – A set of attributes (one or more) that collectively identifies an entity in an entity set.

Candidate Key – A minimal super key is called a candidate key.

An entity set may have more than one candidate key.

Primary Key – A primary key is one of the candidate keys chosen by the database designer to uniquely identify the entity set.

Relationship

A relationship type represents the association between entity types. For example, 'Enrolled in' is a relationship type that exists between entity Student and Course. In ER diagram, relationship is represented by a diamond and connecting the entities with lines.

Degree of a relationship

The number of different entity sets participating in a relationship set is called as degree of a relationship set.

Unary Relationship- When there is only ONE entity set participating in a relation, the relationship is called as unary relationship. For example, an employee can manage another employee.

Binary Relationship - When there are TWO entities set participating in a relation, the relationship is called as binary relationship. For example, Student is enrolled in Course.

If the participant entities are 3, the relationship degree is called **ternary** or degree 3 relationship.

Cardinality

The number of times an entity of an entity set participates in a relationship set is known as cardinality. Cardinality can be of different types:

One to one (1-1) – When each entity in each entity set can take part only once in the relationship, the cardinality is one to one.

One to many(1-M)– When entities in one entity set can take part only once in the relationship set and entities in other entity set can take part more than once in the relationship set, cardinality is many to one.

Many to many (M-M) – When entities in all entity sets can take part more than once in the relationship cardinality is many to many.

The **participation constraint** specifies whether the existence of an entity depends on its being related to another entity via the relationship type. This constraint specifies the minimum number of relationship instances that each entity can participate in and is sometimes called the minimum cardinality constraint. There are two types of participation constraints, **total and partial**.

3.4 Mapping ER-models to relational tables

1. For each strong entity type E, create a relation R that includes all the attributes of E.
2. Choose one of the key attributes of E as the primary key for R. For each weak entity type W with owner entity type E (which is a strong entity), create a relation R & include all attributes of 'W' as attributes of R. Also, include the primary key attribute of the relation that correspond to the owner entity type as foreign key attributes of R .
3. For each binary 1:1 relationship type R in the ER schema, identify the relations S and T that correspond to the entity types participating in R. Foreign Key approach: Choose one of the relations-say S-and include a foreign key in S the primary key of T. It is better to choose an entity type with total participation in R in the role of S.
4. For each binary 1:N relationship type R, identify the relation S that represent the participating entity type at the N-side of the relationship type. Include the primary key of the relation T as foreign key in S that represents the other entity type participating in R. Include any attributes of the 1:N relation type as attributes of S.
5. For each n-ary relationship type R, where $n > 2$, create a new relationship S to represent R. Include the primary keys of the relations that represent the participating entity types as foreign key attributes in S. Also include any attributes of the n-ary relationship type as attributes of S.

3.5 Enhanced Entity Relationship (EER) Model

The complexity of the data is increasing so it becomes more and more difficult to use the traditional ER model for database modeling. To reduce this complexity of modeling we have to make improvements or enhancements to the existing ER model to make it able to handle the complex application in a better way.

Enhanced ER (EER) model is a database model that incorporates the concepts of class/subclass relationships and type inheritance into the ER model. The EER model includes all the modeling

concepts of the ER model. In addition, it includes the concepts of subclass and superclass, categories (UNION types), attribute and relationship inheritance. EER is created to design more accurate database schemas.

Subclasses and **super classes** can be thought as useful ways to describe similarities in the entities represented. Because subclasses inherit their attributes and relationships from the superclass, the database designer can represent complex, hierarchical interdependencies (**inheritance**) within the system being described.

Specialization is the process of dividing a parent-level entity (super-class) into narrower categories (sub-classes) accordingly to all the possible child categories. Specialization requires the separation of entities based on certain uncommon attributes. The process of extracting common characteristics from two or more classes (sub-classes) and combining them into a generalized superclass is called **Generalization**. It is the process that does the grouping entities into broader or distributed categories based on certain common attributes.

Union

Category represents a single super class or sub class relationship with more than one super class. It can be a total or partial participation.

3.6 The Relational Database Model

The relational model represents the database as a collection of relations. A table is represented with columns and rows. Each table is an entity in the ER diagram. Each row is known as a tuple. Each table of the column has a name or attribute of the entity.

3.7 Relational Model Concepts in DBMS

Attribute: Each column in a Table. Attributes are the properties which define a relation

Tables – In the Relational model, relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.

Tuple – It is a single row of a table, which contains a single record.

Column: The column represents the set of values for a specific attribute.

Relation Schema: A relation schema represents the name of the relation with its attributes (columns).

Properties of Relations

- Name of the relation is distinct from all other relations.
- Each relation cell contains exactly one atomic (single) value.
- Each attribute contains a distinct name.
- Each tuple has no duplicate value.
- Order of tuple can have a different sequence.

3.8 Relational Constraints

Relational constraints in DBMS are referred to conditions which must be present for a valid relation. These Relational constraints in DBMS are derived from the rules in the system that the database represents. There are many types of Constraints in DBMS.

Domain Constraints - Every domain (cell in the table or attribute value) must contain atomic values (smallest indivisible units). We perform data type check here, which means we need to assign a data type to a column and limit the values that it can contain.

Key Constraints- These are called uniqueness constraints since it ensures that every tuple in the relation should be unique. An attribute that can uniquely identify a tuple in a relation is called the key or primary key of the table. The value of the attribute for different tuples in the relation has to be unique. Primary keys must be unique and cannot be null.

Referential Integrity Constraints - The Referential integrity constraints is specified between two relations or tables and used to maintain the consistency among the tuples in two relations. Referential Integrity constraints in DBMS are based on the concept of Foreign Keys. A foreign key is an important attribute of a relation which should be referred to in other relationships. Referential integrity constraint state happens where relation refers to a key attribute of a different or same relation. However, that key element must exist in the table.

Chapter Four

Functional Dependency and Normalization

4.1 Functional Dependency

A functional dependency is a constraint between two attributes or sets of attributes of entities in a database. A functional dependency is denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of relation R . The constraint is that, the values of Y component of a tuple in r depend on, or are determined by the values of X component. Alternatively, the values of X component of a tuple uniquely (or functionally) determine the values of Y component.

4.2 Types of Functional Dependency

Partial Dependency - If an attribute which is not a member of the key is dependent on some part of the key (if we have composite/candidate key) then that attribute is partially functionally dependent on the key.

Let $\{A, B\}$ is the key set and C is no key attribute, then if $\{A, B\} \rightarrow C$ and $[B \rightarrow C \text{ or } A \rightarrow C]$ holds. We say C is partially functionally dependent on $\{A, B\}$.

Partial dependency occurs when a non-prime attribute is functionally dependent on part of a candidate/composite key.

Full Dependency - An attribute is fully functionally dependent on another attribute, if it is functionally dependent on that attribute and not on any of its proper subset. For example, an attribute Q is fully functionally dependent on another attribute P , if it is functionally dependent on P and not on any of the proper subset of P .

Let $\{A, B\}$ is the key set and C is not key attribute, then if $\{A, B\} \rightarrow C$ and $[B \rightarrow C \ \& \ A \rightarrow C]$ doesn't hold (i.e; if A can't determine C and B can't determine C), then C is fully functionally dependent on $\{A, B\}$.

Transitive Dependency - In mathematics and logic, a transitive relationship is a relationship of the following form: "If $A \rightarrow B$, and if also $B \rightarrow C$, then $A \rightarrow C$."

Example: If $X > Y$ and $Y > Z$ then $X > Z$.

Generalized way of describing transitive dependency is that:

If A functionally governs B, AND If B functionally governs C THEN 'A' functionally governs 'C', provided that neither C nor B determines A. A functional dependency is said to be transitive if it is indirectly formed by two functional dependencies.

4.3 Database Anomalies

Database Anomalies are the flaws in databases which occurs because of poor database design and storing everything in a single flat database. They are problems that are caused usually by redundant data. Types of database anomalies are Insertion anomaly, Deletion anomaly, and Update anomaly.

Insertion Anomaly - occurs when the entire primary key is not known and the database cannot insert a new record properly. This would violate entity integrity. It can be avoided by using a sequence number for the primary key.

Deletion Anomaly - happens when a record is deleted that results in the loss of other related data. It can be avoided by normalizing tables in a database to minimize their dependency.

Update Anomaly - occurs when a change to one attribute value causes the database to either contain inconsistent data or causes multiple records to need changing. It may be prevented by making sure tables are in third normal form.

4.4 Normalization

Normalization is a process of organizing the data in database to avoid data redundancy, insertion anomaly, update anomaly & deletion anomaly. It is a technique of organizing the data in the database. It is a systematic approach of decomposing tables to eliminate data redundancy (repetition) and undesirable characteristics like Insertion, Update and Deletion Anomalies. It is a multi-step process that puts data into tabular form, removing duplicated data from the relation tables.

Normal forms

Redundancy in a relation may causes insertion, deletion, and update anomalies. Normalization is database design technique that minimizes redundancy from a relation or a set of relations. Normalization rules divides larger tables into smaller tables and links them using relationship. Normal Forms are maintenance rules that are used to eliminate or reduce the redundancy in database relations. These normal forms are 1NF (First Normal Form), 2NF (Second Normal Form) and 3NF (Third Normal Form).

First normal form (1NF)

A relation (table) R is in 1NF if and only if (iff) all underlying domains of attributes contain only atomic (simple or indivisible) values. i.e. The value of any attribute in a tuple (row) must be a single value from the domain of their attribute. It is defined to disallow multivalued attributes, composite attributes, and their combinations. In other words, 1NF disallows relations within relations or relations as attribute values within tuples. It must be remove repeating groups (values) in a single field. The primary key with repeating group attributes are moved into a new table. When a relation contains no repeating groups (values), it is in first normal form (1NF).

- Example1 (unnormalized)

DEPARTMENT			
Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

- Normalized (1st NF)

DEPARTMENT			
Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Second normal form (2NF)

Second normal form (2NF) is based on the concept of full functional dependency. A relation schema R is in 2NF if it is in 1NF and every non-prime attribute A in R is fully functionally dependent on the primary key (i.e. the non-prime attributes are not partially dependent on key attributes). Remove any partially dependent attributes and place them in another relation. A partial dependency is when the attribute(s) is/are dependent on a part of a key.

Example: Consider a relation schemas for Employees and Teams in a single relation as follows

Emp-Teams(EmpId, Name, BDate, Gender, TeamId, Project, TeamName)

Then upon decomposition we will have

$\text{EmpId} \rightarrow \{\text{Name}, \text{BDate}, \text{Gender}\}$

$\text{TeamId} \rightarrow \{\text{Project}, \text{TeamName}\}$

2nd normal form will be

Employees(EmpId, Name, BDate, Gender)

Teams(TeamId, Project, TeamName)

Emp-Teams(EmpId, TeamId)

Third Normal Form (3NF)

A relation schema R is in third normal form (3NF) if it is in 2NF and every non-prime attribute of 'R' are fully functionally dependent on every key of R and non-transitively dependent on every key of R.

emp_id	emp_name	emp_zip	emp_state	emp_city	emp_district
1001	John	282005	UP	Agra	Dayal Bagh
1002	Ajeet	222008	TN	Chennai	M-City
1006	Lora	282007	TN	Chennai	Urrapakkam
1101	Lilly	292008	UK	Pauri	Bhagwan
1201	Steve	222999	MP	Gwalior	Ratan

employee table:

emp_id	emp_name	emp_zip
1001	John	282005
1002	Ajeet	222008
1006	Lora	282007
1101	Lilly	292008
1201	Steve	222999

After normalization

employee_zip table:

emp_zip	emp_state	emp_city	emp_district
282005	UP	Agra	Dayal Bagh
222008	TN	Chennai	M-City
282007	TN	Chennai	Urrapakkam
292008	UK	Pauri	Bhagwan
222999	MP	Gwalior	Ratan

Chapter Five

Record Storage and Primary File Organization

5.1 Introduction

To translate the logical description of data into technical specifications for storing and retrieving data, a physical database design is needed. The goal is to create a design for storing and retrieving data that will provide adequate performance and insure database integrity, security, and availability.

Some decisions need to be made on the physical database design. They are Attribute data types, data/file storages, File organizations and Indexes.

5.2 Choosing Attribute Data Types

CHAR - fixed-length character

VARCHAR() - variable-length characters

DATE – actual date

BLOB – binary large object (good for graphics images, audio, sound clips, etc.)

NUMERIC - positive/negative numbers

Exact

- SMALLINT → 16 bit
- INT → 32 bit
- LONG → 64 bit
- DECIMAL → 128 bit ...

Approximate

- FLOAT (single precision) → 32 bit with $\approx$ 6-9 digit precision...
- REAL (variation of double) → 32 bit with $\approx$ 15 digit precision...
- DOUBLE (double precision) → 64 bit with $\approx$ 32 bit precision...

5.3 Data/File Storage: Categories

Three main storage categories: Primary storage, Secondary storage, and Tertiary storage.

Primary storage - includes storage media that store a data which can be operated directly by the computer central processing unit (CPU). Primary storage includes; the computer main memory (RAM) and Smaller but faster cache memories.

Primary storage usually provides fast read and write to data. It is of limited storage capacity. Primary storage devices are more expensive. The contents of main memory are lost in case maybe when power failure, a system crash or other issue occurs.

Secondary storage - includes large storage devices such as computer hard disk (HDD). These devices usually have a larger capacity, less cost, and Provide slower access to data than primary storage devices.

Data in secondary storage cannot be processed directly by the CPU, it must first be copied into primary storage. They are called online storage devices because they can be accessed in short period of time whenever needed.

Tertiary storage - optical disks (CD-ROMs, DVDs, and other similar storage media) and magnetic tapes which are removable media are used in today's systems as offline storage for archiving databases.

5.4 File Organizations

There are several **primary file organization** types, which determine how the file records are physically placed on the disk.

Heap file (unordered file) - places the records on disk in no particular order → by appending new records at the end of the file.

Sorted file (sequential file) - keeps the records ordered by the value of a particular field (called the sorting key).

Hashed file - uses a hash function applied to a particular field (called the hash key) to determine a record's placement on disk.

Secondary organization or auxiliary access structures allows efficient access & storage to file records based on alternate fields than those that have been used for the primary file organization.

5.5 Index

Indexes are additional auxiliary access structures which are used to speed up the retrieval of records in response to certain search conditions. Index structures are additional files, which provide alternative ways to access the records without affecting the physical placement of records on disk.

Types of Indexes

Dense Index

An index entry is created for every search key value (for each records) in each block. This index contains search key value and a pointer to the actual record. It has large index size. Less time needed to locate arbitrary data.

Sparse Index

One index entry for each block. Indexes are created only for some of the data records. It has small index size and more time needed to locate arbitrary data Records must be clustered or arranged in blocks.

Single Level Ordered Indexes

Here it is assumed that a file already exists with some primary organization like unordered, ordered, or hashed organizations. For a file with a given record structure consisting of several fields, an index access structure is usually defined on a single field of a file, called an indexing field (or indexing attribute). Types of Single Level Ordered Indexes are **Primary, Clustering and Secondary**.

Primary Index

A primary index is specified on the ordering key field of an ordered file of records. An ordering key field is used to physically order the file records on disk, and every record has a unique value for that field. Primary index is an ordered file whose records are of fixed length with two

fields. The first field is of the same data type as the ordering key field called the PK of the data file, and the second field is a pointer to a disk block (block address).

There is one index entry (or index record) in the index file for each block in the data file. Each index entry has the value of the primary key field for the first record in a block and a pointer to that block as its two field values. It is sparse.

The index file for a primary index occupies a much smaller space than the data file does, because there are fewer index entries than the records in the data file and each index entry is typically smaller in size than a data record because it has only two fields, both of which tend to be short in size. To locate a record with a search key value requires $\log_2(b) + 1$

Clustering Indexes

Here files are assumed as ordered with a Non-key field. If file records are physically ordered on a non-key field which doesn't have a distinct value for each record that field is called clustering field and the data file is called a clustered file.

We can create a different type of index, called a clustering index, to speedup retrieval of all records that have the same value for the clustering field. If the ordering field is not a key field i.e; if numerous records in the file can have the same value for the ordering field clustering index can be used.

A clustering index is also an ordered file with two fields. The first field is of the same type as the clustering field of the data file, and the second field is a disk block/cluster pointer.

There is one entry in the clustering index for each distinct value of the clustering field, and it contains the value and a pointer to each block in the data file that has a record with that value for its clustering field. It is a non-dense indexing mechanism.

Secondary index

Provides a secondary means of accessing a data file for which some primary access already exists. The data file records could be ordered, unordered, or hashed. Secondary index may be created on a field that is a candidate key and has a unique value in every record, or on a non-key field with duplicate values.

A data file can have several secondary indexes in addition to its primary access method. The index is again an ordered file with two fields. The first field is of the same data type as some non-ordering field of the data file that is an indexing field. The second field is either a block or a record pointer.

Many secondary indexes can be created for the same file each represents an additional means of accessing that file based on some specific field.

Hence, such an index is dense. A secondary index usually needs more storage space and longer search time than does a primary index, because of its larger number of entries. However, the improvement in search time for an arbitrary record is much greater for a secondary index than for a primary index.

Chapter Six

The Relational Algebra and Relational Calculus

6.1 Relational algebra

Relational algebra is the basic set of operations for the relational model. These operations enable a user to specify basic retrieval requests (or queries). The result of an operation is a new relation, which may have been formed from one or more input relations. The algebra operations thus produce new relations. Since each operation returns a relation, operations can be composed (Algebra is “closed”).

Basic operations:

Selection (σ) - Selects a subset of rows from relation.

Projection (π) - Deletes unwanted columns from relation.

Cross-product ($\times$) - Allows us to combine two relations.

Set-difference ($-$) - Tuples in relation 1, but not in relation 2.

Union ($\cup$) - Tuples in relation 1 and in relation 2.

Intersection ($\cap$) - Only tuples in relation 1 and in relation 2 (only the common tuples).

6.2 Relational Calculus

A relational calculus expression creates a new relation, which is specified in terms of variables that range over rows of the stored database relations (in tuple calculus) or over columns of the stored relations (in domain calculus).

In a calculus expression, there is no order of operations to specify how to retrieve the query result. A calculus expression specifies only what information the result should contain. This is the main distinguishing feature between relational algebra and relational calculus. Relational calculus is considered to be a nonprocedural language. This differs from relational algebra, where we must write a sequence of operations to specify a retrieval request; hence relational algebra can be considered as a procedural way of stating a query.

Tuple Relational Calculus

The tuple relational calculus is based on specifying a number of tuple variables. Each tuple variable usually ranges over a particular database relation, meaning that the variable may take as its value any individual tuple from that relation.

A simple tuple relational calculus query is of the form

$\{t \mid \text{COND}(t)\}$ where t is a tuple variable and $\text{COND}(t)$ is a conditional expression involving t . The result of such a query is the set of all tuples t that satisfy $\text{COND}(t)$.

Chapter Seven - SQL

7.1 SQL

SQL represents Structured Query Language. SQL is a comprehensive database language: It has statements for data definitions, and updates. Hence, it is both a DDL (Data Definition language) and a DML (Data Manipulation Language).

7.1.1 DDL (data definition language)

Basic DDL commands we usually use are: CREATE, DROP, ALTER, TRUNCATE and RENAME

7.1.2 DML (data manipulation language)

Basic DDL commands are: SELECT, PROJECT, JOIN, INSERT, DELETE, and UPDATE

Chapter 1: Query Processing and Optimization

1.5 Query Processing and Optimization: Why?

What is Query Processing?

Steps required to **transform high level SQL** query into a correct and “efficient” **strategy for execution and retrieval**.

What is Query Optimization?

- ✓ The activity of choosing a single “efficient” execution strategy (from hundreds) as determined by database catalog statistics.
- ✓ The aim is to answer which relational algebra expression, equivalent to the given query, will lead to the most efficient solution plan?
- ✓ For each algebraic operator, what algorithm (of several available) do we use to compute that operator?
- ✓ How do operations pass data (main memory buffer, disk buffer,...)?
- ✓ Will this plan minimize resource usage? (CPU/Response Time/Disk)

Example:

Identify all managers who work in a London branch

SELECT *

FROM Staff s, Branch b

WHERE s.branchNo = b.branchNo AND s.position = 'Manager' AND
b.city = 'london';

Results in these equivalent relational algebra statements

(1) $S(\text{position}='Manager') \wedge (\text{city}='London') \wedge (\text{Staff.branchNo}=\text{Branch.branchNo})$ (**Staff X Branch**)

(2) $S(\text{position}='Manager') \wedge (\text{city}='London')$ (**Staff** Staff.branchNo = Branch.branchNo **Branch**)

(3) $[S(\text{position}='Manager') (\text{Staff})] \text{ wv } \text{Staff.branchNo} = \text{Branch.branchNo} [S(\text{city}='London') (\text{Branch})]$

Assume:

- 1000 tuples in **Staff**.
- 50 Managers
- 50 tuples in **Branch**.
- 5 London branches

- No indexes or sort keys
- All temporary results are written back to disk (memory is small)
- Tuples are accessed one at a time (not in blocks)

$(\text{Position}='Manager') \wedge (\text{city}='London') \wedge (\text{Staff.branchNo}=\text{Branch.branchNo})$ (**Staff X Branch**)

- Requires $(1000+50)$ disk accesses to read from Staff and Branch relations
- Creates temporary relation of Cartesian Product $(1000*50)$ tuples
- Requires $(1000*50)$ disk access to read in temporary relation and test predicate

Total Work = $(1000+50) + 2*(1000*50) =$

101,050 I/O operations

Query 1 (Bad)

$\square (\text{position}='Manager') \wedge (\text{city}='London') \wedge (\text{Staff.branchNo}=\text{Branch.branchNo})$ (**Staff X Branch**)

- Requires $(1000+50)$ disk accesses to read from Staff and Branch relations
- Creates temporary relation of Cartesian Product $(1000*50)$ tuples
- Requires $(1000*50)$ disk access to read in temporary relation and test predicate

Total Work = $(1000+50) + 2*(1000*50) = 101,050$ I/O operations

Query 2 (Better)

$\square (\text{position}='Manager') \wedge (\text{city}='London')$ (**Staff** $\square \square$ $\text{Staff.branchNo} = \text{Branch.branchNo}$ **Branch**)

- Again requires $(1000+50)$ disk accesses to read from **Staff** and **Branch**
- Joins **Staff** and **Branch** on branchNo with 1000 tuples
(1 employee : 1 branch)
- Requires (1000) disk access to read in joined relation and check predicate

Total Work = $(1000+50) + 2*(1000) =$

3050 I/O operations

3300% Improvement over Query 1

Query 3 (Best)

$[\square (\text{position}='Manager') (\text{Staff})] \square \square \text{Staff.branchNo} = \text{Branch.branchNo} [\square (\text{city}='London') (\text{Branch})]$

- Read **Staff** relation to determine 'Managers' (1000 reads)
 - Create 50 tuple relation (50 writes)

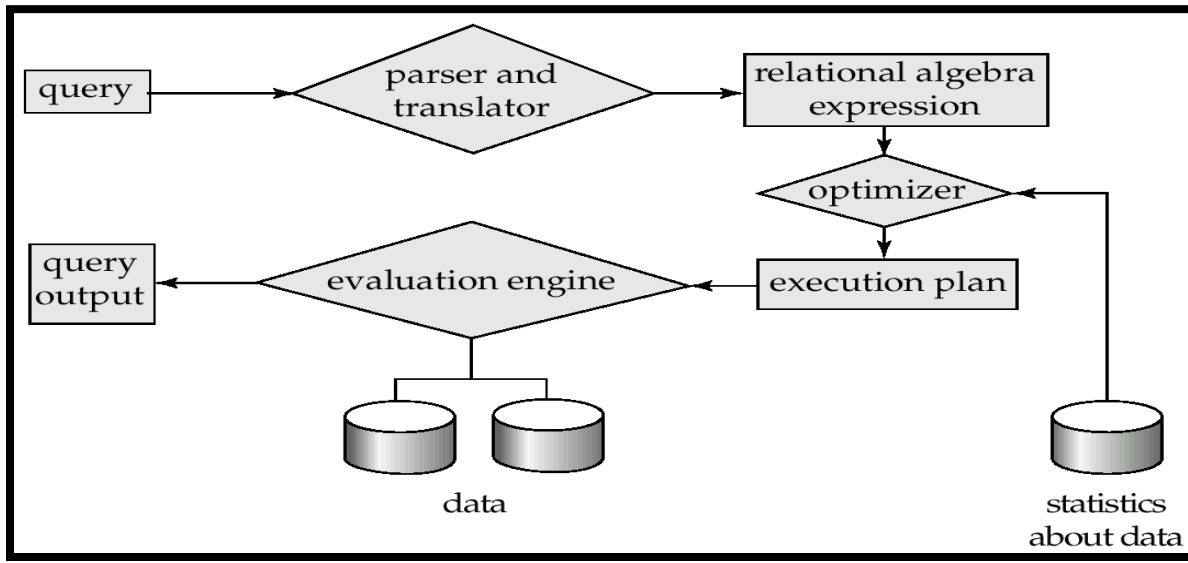
- Read *Branch* relation to determine 'London' branches (50 reads)
 - Create 5 tuple relation(5 writes)
- Join reduced relations and check predicate (50 + 5 reads)

Total Work = 1000 + 2*(50) + 5 + (50 + 5) =

1160 I/O operations

8700% Improvement over Query 1

1.6 Query Processing Steps



- Processing can be divided into 4 :Decomposition, Optimization , Execution and Code generation

1.7 Query Decomposition

- It is the process of transforming a high level query into a relational algebra query, and to check that the query is syntactically and semantically correct. It Consists of parsing and validation

Typical stages in query decomposition are:

- Analysis: lexical and syntactical analysis of the query (correctness) based on attributes, data type.. .. Query tree will be built for the query containing leaf node for base relations, one or many non-leaf nodes for relations produced by relational algebra operations and root node for the result of the query. Sequence of operation is from the leaves to the root.(SELECT * FROM Catalog c ,Author a Where a.authorid = c.authorid AND c.price>200 AND a.country= ' USA').

- **Key activities**
 - **Analyze query lexically and syntactically using compiler techniques.**
 - **Verify relations and attributes exist.**
 - **Verify operations**
- ii. Normalization: convert the query into a normalized form. The predicate WHERE will be converted to Conjunctive ($\wedge$) or Disjunctive ($\vee$) Normal form.
- iii. Semantic Analysis: to reject normalized queries that are not correctly **formulated** or **contradictory**. Incorrect if components do not contribute to generate result. Contradictory if the predicate can not be satisfied by any tuple. Say for example, (Catalog = "BS" $\wedge$ Catalog = "CS") since a given book can only be classified in either of the category at a time
- iv. Simplification: to detect redundant qualifications, eliminate common sub-expressions, and transform the query to a semantically equivalent but more easily and effectively computed form. For example, If a user don't have the necessary access to all of the objects of the query , it should be rejected.

1.8 Query Optimization

- Everyone wants the performance of their database to be optimal. In particular, there is often a requirement for a specific query or object that is query based, to run faster.
- Problem of query optimization is to **find the sequence of steps** that produces the answer to user request in the **most efficient manner**, given the database structure.
- The performance of a query is affected by the tables or views that underlies the query and by the complexity of the query.
- Given a request for data manipulation or retrieval, an **optimizer will choose an optimal plan** for evaluating the request from among the manifold alternative strategies. i.e. there are many ways (access paths) for accessing desired file/record.
- Hence ,DBMS is responsible to pick the best execution strategy based on various considerations(**Least amount of I/O and CPU resources.**)
- Example: Consider relations $r(AB)$ and $s(CD)$. We require $r \bowtie s$.
- Method 1 :
 - a. Load next record of r in RAM.
 - b. Load all records of s , one at a time and concatenate with r .

c. All records of *r* concatenated?

- NO: goto a.
- YES: exit (the result in RAM or on disk).
- Performance: Too many accesses.
- Method 2: Improvement

a. Load as many blocks of *r* as possible leaving room for one block of *s*.

b. Run through the *s* file completely one block at a time.

- Performance: Reduces the number of times *s* blocks are loaded by a factor of equal to the number of *r* records than can fit in main memory.
- Considerations during query Optimization:
 - **Narrow down intermediate result sets quickly.** SELECT and PROJECTION before JOIN
 - **Use access structures** (indexes).

Approaches to Query Optimization: Heuristics and Cost Function

A. Heuristics Approach

- Heuristics Approach uses **the knowledge of the characteristics of the relational algebra operations and the relationship** between the operators to optimize the query.
- Thus the heuristic approach of optimization will make use of:
 - Properties of individual operators
 - Association between operators
 - Query Tree: a graphical representation of the operators, relations, attributes and predicates and processing sequence during query processing.
- **It is composed of three main parts:**
 - The Leafs: the **base relations** used for processing the query/ extracting the required information
 - The Root: the **final result/relation** as an output based on the operation on the relations used for query processing

- Nodes: intermediate results or relations before reaching the final result.
- **Sequence of execution of operation in a query tree will start from the leaves and continues to the intermediate nodes and ends at the root.**

Transformation Rules for Relational Algebra

1. Cascade of SELECTION: conjunctive SELECTION Operations can cascade into individual Selection Operations and Vice Versa

$$\sigma_{(c1 \wedge c2 \wedge c3)}(R) = \sigma_{c1}(\sigma_{c2}(\sigma_{c3}(R))) \text{ where } c_i \text{ is a predicate}$$

2. Commutativity of SELECTION operations

$$\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c2}(\sigma_{c1}(R)) \text{ where } c_i \text{ is a predicate}$$

3. Cascade of PROJECTION: in the sequence of PROJECTION Operations, only the last in the sequence is required

$$\pi_{L1} \pi_{L2} \pi_{L3} \pi_{L4}(R) = \pi_{L1}(R)$$

4. Commutativity of SELECTION with PROJECTION and Vice Versa
 - If the predicate $c1$ involves only the attributes in the projection list ($L1$), then the selection and projection operations commute

$$\pi_{L1}(\sigma_{c1}(R)) = \sigma_{c1}(\pi_{L1}(R))$$

5. Commutativity of THETA JOIN/Cartesian Product
 $R \times S$ is equivalent to $S \times R$
 Also holds for Equi-Join and Natural-Join

$$(R \bowtie_{c1} S) = (S \bowtie_{c1} R) \text{ or } (R \times S) = (S \times R)$$

6. Commutativity of SELECTION with THETA JOIN
 - a. If the predicate $c1$ involves only attributes of one of the relations (R) being joined, then the Selection and Join operations commute

$$\sigma_{c1}(R \bowtie_c S) = (\sigma_{c1}(R)) \bowtie_c S$$

- b. If the predicate is in the form $c1 \wedge c2$ and $c1$ involves only attributes of R and $c2$ involves only attributes of S, then the Selection and Theta Join operations commute

$$\sigma_{c1 \wedge c2}(R \bowtie_c S) = (\sigma_{c1}(R)) \bowtie_c (\sigma_{c2}(S))$$

7. Commutativity of PROJECTION and THETA JOIN

If the projection list is of the form L_1, L_2 , where L_1 involves only attributes of R and L_2 involves only attributes of S being joined and the predicate c involves only attributes in the projection list, then the SELECTION and JOIN operations commute

$$\pi_{L_1, L_2} (R \bowtie_c S) = (\pi_{L_1} (R)) \bowtie_c (\pi_{L_2} (S))$$

8. Commutativity of the Set Operations: UNION and INTERSECTION but not SET DIFFERENCE

$$R \cap S = S \cap R \text{ and } R \cup S = S \cup R$$

9. Associativity of the THETA JOIN, CARTESIAN PRODUCT, UNION and INTERSECTION.

$$(R \theta S) \theta T = R \theta (S \theta T) \text{ where } \theta \text{ is one of the operations}$$

10. Commuting SELECTION with SET OPERATIONS

$$\sigma_c (R \theta S) = (\sigma_c (R) \theta \sigma_c (S)) \text{ where } \theta \text{ is one of the operations}$$

11. Commuting PROJECTION with UNION

$$\pi_{L_1} (S \cup R) = \pi_{L_1} (S) \cup \pi_{L_1} (R)$$

Heuristic Approach will be implemented by using the above transformation rules in the following sequence or steps.

Sequence for Applying Transformation Rules

1. Use
Rule-1 → Cascade SELECTION
2. Use
Rule-2: Commutativity of SELECTION
Rule-4: Commuting SELECTION with PROJECTION
Rule-6: Commuting SELECTION with JOIN and CARTESIAN
Rule-10: commuting SELECTION with SET OPERATIONS
3. Use
Rule-9: Associativity of Binary Operations (JOIN, CARTESIAN, UNION and INTERSECTION). Rearrange nodes by making the most restrictive operations to be performed first (moving it as far down the tree as possible)
4. Perform Cartesian Operations with the subsequent Selection Operation
5. Use
Rule-3: Cascade of PROJECTION
Rule-4: Commuting PROJECTION with SELECTION
Rule-7: Commuting PROJECTION with JOIN and CARTESIAN
Rule-11: commuting PROJECTION with UNION

Using Heuristics in Query Optimization

- Process for heuristics optimization
 1. The parser of a high-level query generates an initial internal representation;
 2. Apply heuristics rules to optimize the internal representation.
 3. A query execution plan is generated to execute groups of operations based on the access paths available on the files involved in the query.
- The main heuristic is to apply first the operations that reduce the size of intermediate results.
 1. E.g. Apply SELECT and PROJECT operations before applying the JOIN or other binary operations.
- **Query block:** The basic unit that can be translated into the algebraic operators and optimized.
- A query block contains a single SELECT-FROM-WHERE expression, as well as GROUP BY and HAVING clause if these are part of the block.
- **Nested queries** within a query are identified as separate query blocks.
- **Query tree:**
 1. A tree data structure that corresponds to a relational algebra expression. It represents the input relations of the query as **leaf nodes** of the **tree**, and represents the relational algebra operations as internal nodes.
- An execution of the query tree consists of executing an internal node operation whenever its operands are available and then replacing that internal node by the relation that results from executing the operation.
- **Query graph:**
 1. A graph data structure that corresponds to a relational calculus expression. It does *not* indicate an order on which operations to perform first. There is only a *single* graph corresponding to each query.
- **Example:**
 - For every project located in 'Stafford', retrieve the project number, the controlling department number and the department manager's last name, address and birthdate.
- Relation algebra:

- ☐ PNUMBER, DNUM, LNAME, ADDRESS, BDATE
 (((☐ PLOCATION='STAFFORD',**(PROJECT))**
(DEPARTMENT)) DNUM=DNUMBER MGRSSN=SSN
(EMPLOYEE))
- SQL query:
 Q2: SELECT P.NUMBER,P.DNUM,E.LNAME,E.ADDRESS, E.BDATE
 FROM PROJECT AS P,DEPARTMENT AS D, EMPLOYEE AS
 E
 WHERE P.DNUM=D.DNUMBER AND
 D.MGRSSN=E.SSN AND
 P.PLOCATION='STAFFORD';

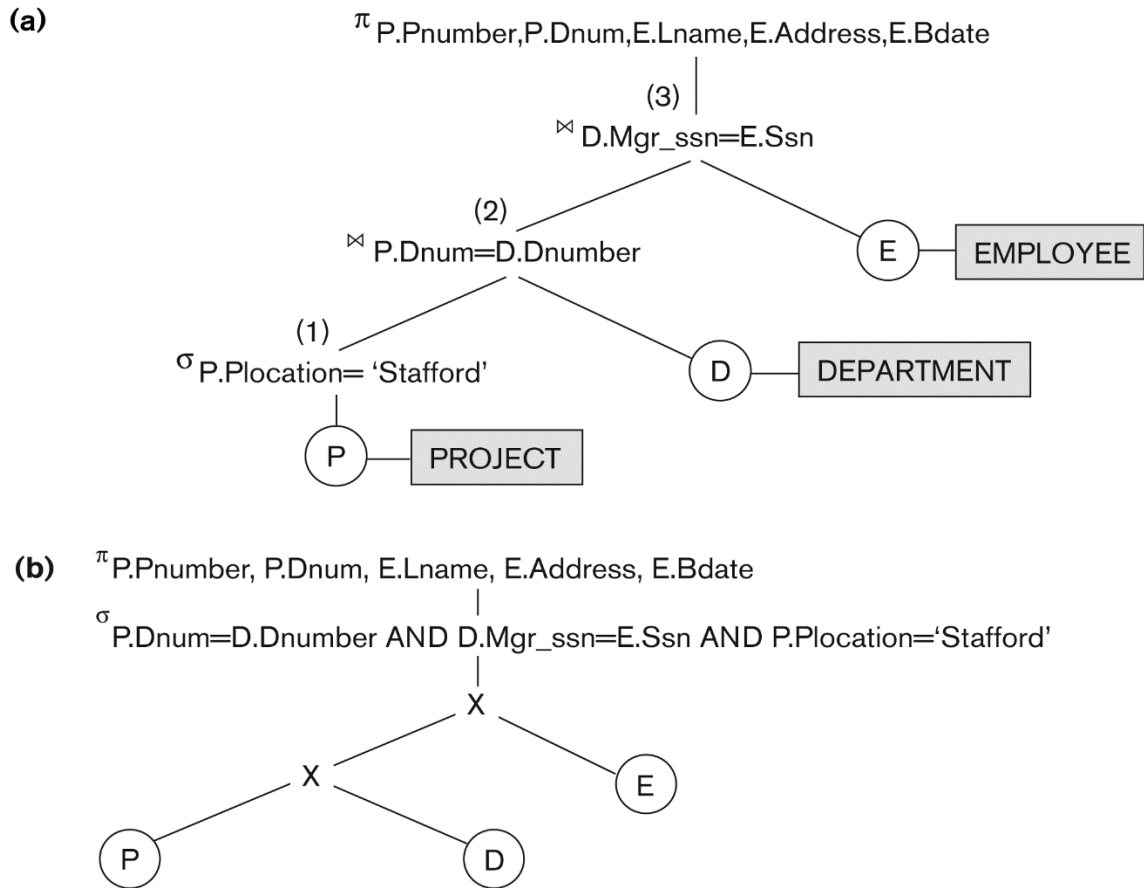


Figure 15.4

Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

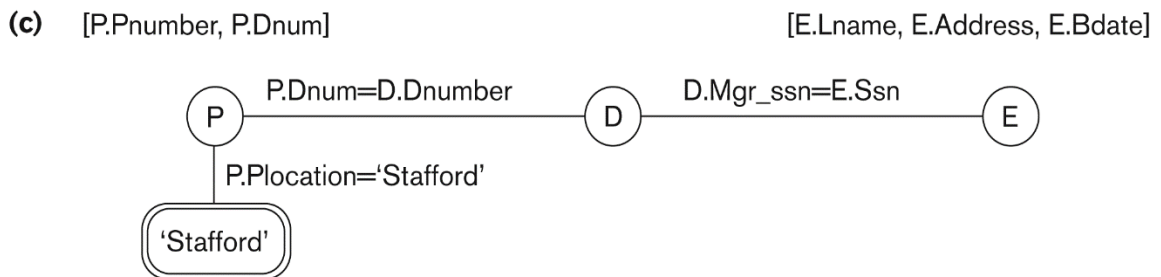


Figure 15.4

Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

- Heuristic Optimization of Query Trees:
 - The same query could correspond to many different relational algebra expressions — and hence many different query trees.

- Example:

AND BDATE >

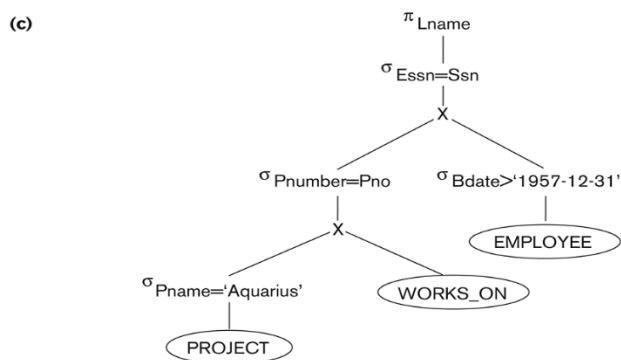
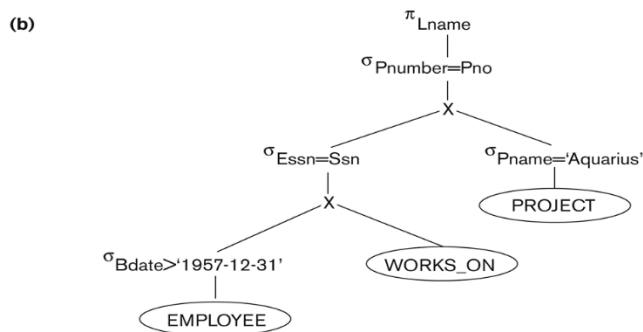
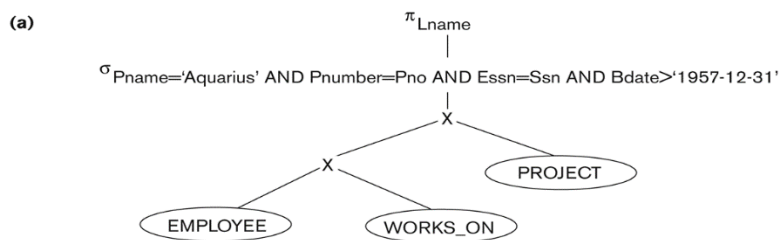
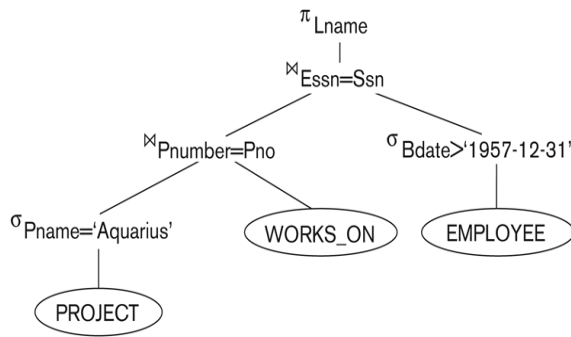


Figure 15.5
Steps in converting a query tree during heuristic optimization.

- (a) Initial (canonical) query tree for SQL query Q.
- (b) Moving SELECT operations down the query tree.
- (c) Applying the more restrictive SELECT operation first.
- (d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.
- (e) Moving PROJECT operations down the query tree.

(d)



(e)

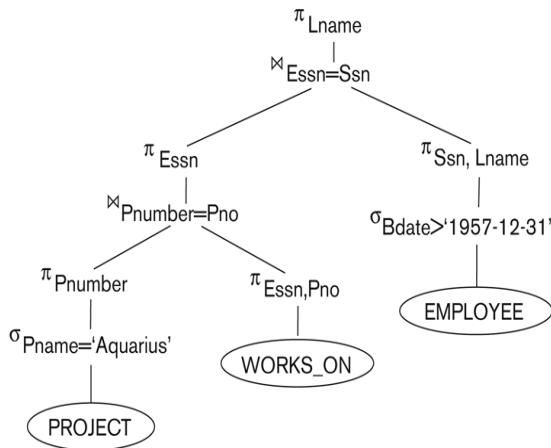


Figure 15.5

Steps in converting a query tree during heuristic optimization.

(a) Initial (canonical) query tree for SQL query Q.

(b) Moving SELECT operations down the query tree.

(c) Applying the more restrictive SELECT operation first.

(d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.

(e) Moving PROJECT operations down the query tree.

Transformation Rules for RA Operations

• For example:

$$\begin{aligned}
 & (\text{Staff} \bowtie_{\text{Staff.staffNo}=\text{PropertyForRent.staffNo}} \bowtie_{\text{P.ownerNo}=\text{Owner.ownerNo} \wedge \text{staff.IName}=\text{Owner.IName}} \text{PropertyForRent}) \\
 & \quad \text{Owner} \\
 & = \\
 & \text{Staff} \bowtie_{\text{staff.staffNo}=\text{PropertyForRent.staffNo} \wedge \text{staff.IName}=\text{IName}} (\text{PropertyForRent} \bowtie_{\text{ownerNo}} \text{Owner})
 \end{aligned}$$

Use of Transformation Rules

For prospective renters of flats, find properties that match requirements and owned by CO93.

SELECT p.propertyNo, p.street

FROM Client c, Viewing v, PropertyForRent p

WHERE c.prefType = 'Flat' AND

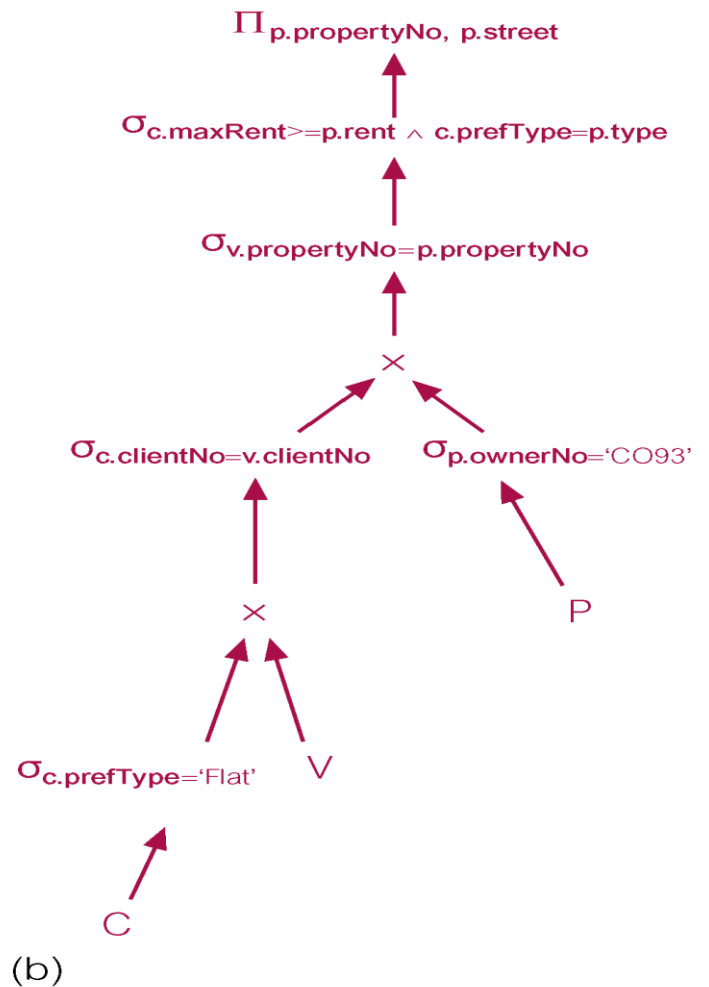
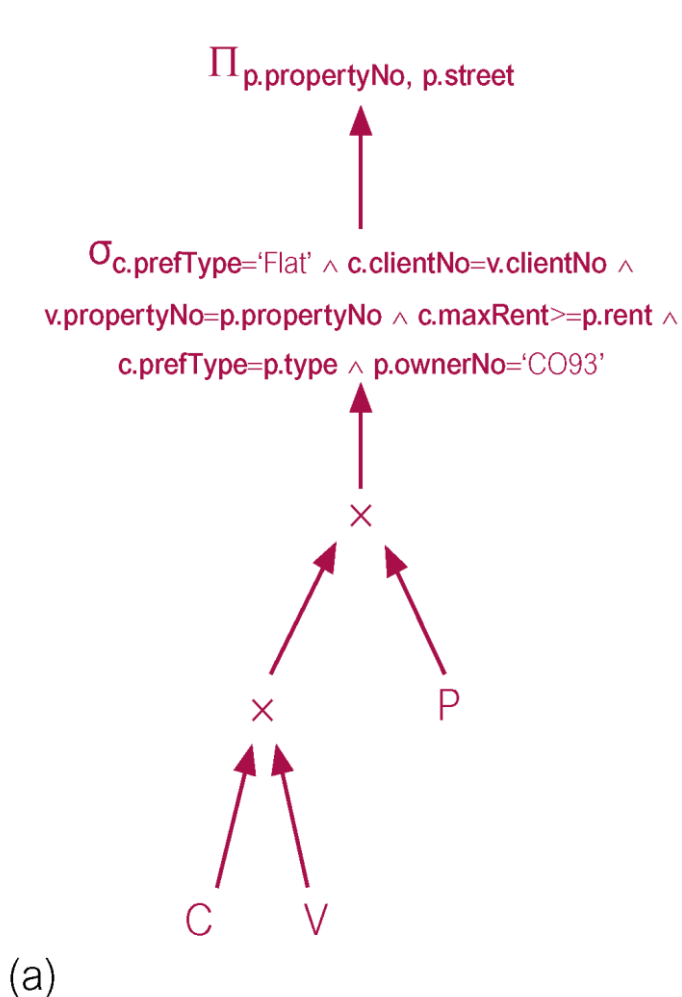
c.clientNo = v.clientNo AND

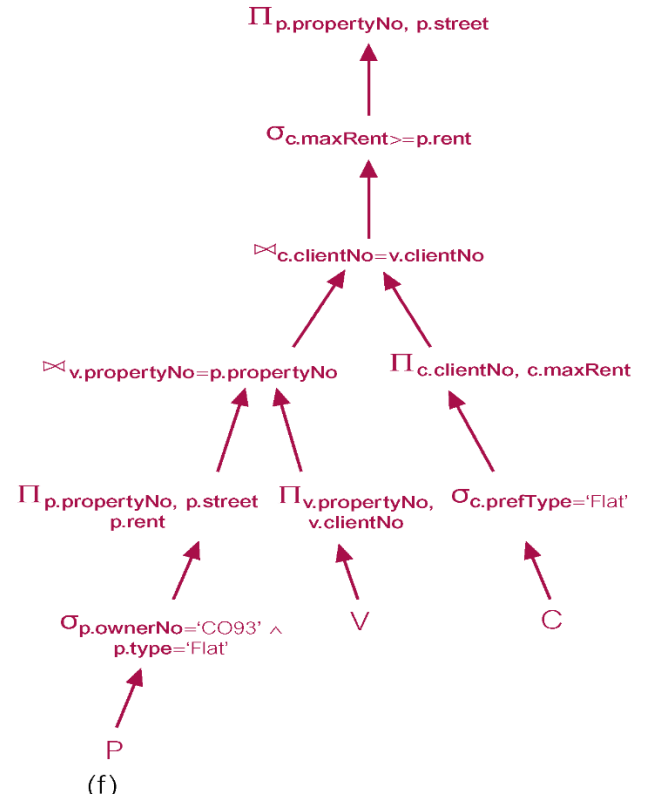
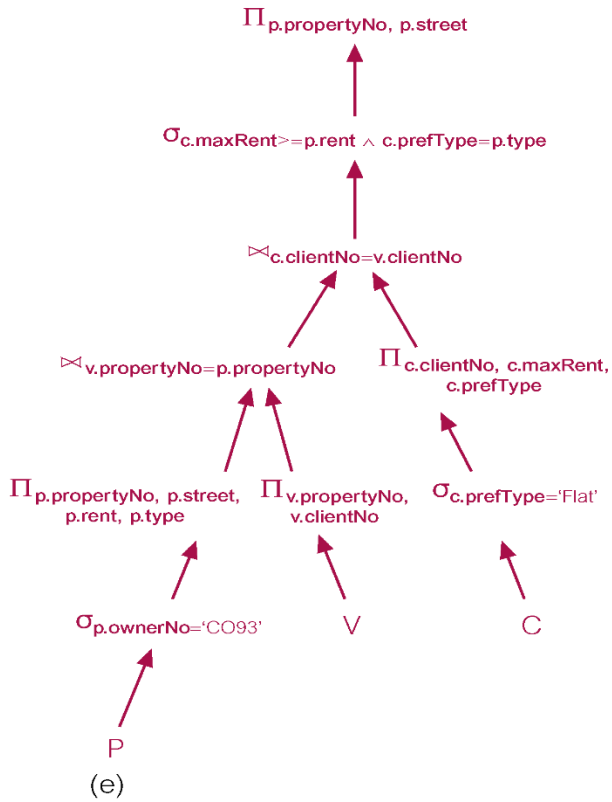
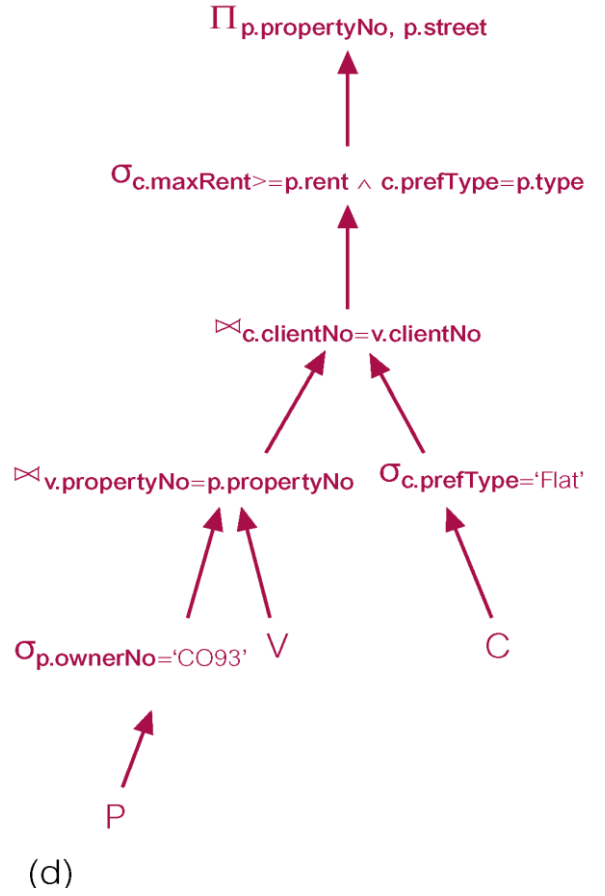
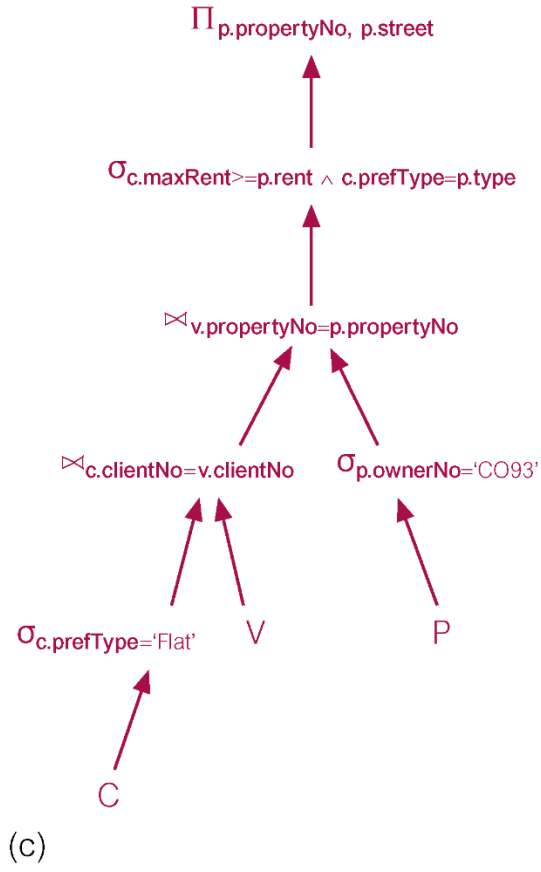
v.propertyNo = p.propertyNo AND

c.maxRent >= p.rent AND

c.prefType = p.type AND

p.ownerNo = 'CO93';





Heuristical Processing Strategies

- Perform Selection operations as early as possible.
 - Keep predicates on same relation together.
- Combine Cartesian product with subsequent Selection whose predicate represents join condition into a Join operation.
- Use associativity of binary operations to rearrange leaf nodes so leaf nodes with most restrictive Selection operations executed first.
- Perform Projection as early as possible.
- Keep projection attributes on same relation together.
- Compute common expressions once.
- If common expression appears more than once, and result not too large, store result and reuse it when required.
- Useful when querying views, as same expression is used to construct view each time.

Summary of Heuristics for Algebraic Optimization:

1. The main heuristic is to apply first the operations that **reduce the size of intermediate results**.
2. Perform select operations as early as possible to **reduce the number of tuples** and perform project operations as early as possible to **reduce the number of attributes**. (This is done by moving select and project operations as far down the tree as possible.)
3. The select and join operations that are **most restrictive should be executed before other** similar operations. (This is done by reordering the leaf nodes of the tree among themselves and adjusting the rest of the tree appropriately.)

B. Cost Estimation Approach to Query Optimization

- The main idea is to minimize the cost of processing a query. The cost function is comprised of:
- I/O cost + CPU processing cost + communication cost + Storage cost
- These components might have different weights in different processing environments

- The DBMs will use information stored in the system catalogue for the purpose of estimating cost.
- The main target of query optimization is to minimize the size of the intermediate relation. The size will have effect in the cost of:
 - Disk Access
 - Data Transportation
 - Storage space in the Primary Memory
 - Writing on Disk

Cost Estimation for RA Operations

- Many different ways of implementing RA operations.
- Aim of QO is to choose most efficient one.
- Use formulae that estimate costs for a number of options, and select one with lowest cost.
- Consider only cost of disk access, which is usually dominant cost in QP.
- Many estimates are based on cardinality of the relation, so need to be able to estimate this.

Using Selectivity and Cost Estimates in Query Optimization

- **Cost-based query optimization:**
 - Estimate and compare the costs of executing a query using different execution strategies and choose the strategy with the lowest cost estimate. (Compare to heuristic query optimization)
- Issues
 - Cost function
 - Number of execution strategies to be considered
- Cost Components for Query Execution
 1. Access cost to secondary storage
 2. Storage cost
 3. Computation cost

4. Memory usage cost
5. Communication cost

1. Access Cost of Secondary Storage

- Data is going to be accessed from secondary storage, as a query will be needing some part of the data stored in the database. The disk access cost can again be analyzed in terms of:
 - Searching
 - Reading, and
 - Writing, data blocks used to store some portion of a relation.
- Remark: The disk access cost will vary depending on
 - The file organization used and the access method implemented for the file organization.
 - whether the data is stored contiguously or in scattered manner, will affect the disk access cost.

2. Storage Cost

- While processing a query, as any query would be composed of many database operations, there could be one or more intermediate results before reaching the final output. These intermediate results should be stored in primary memory for further processing. The bigger the intermediate relation, the larger the memory requirement, which will have impact on the limited available space. This will be considered as a cost of storage.

3. Computation Cost

- Query is composed of many operations. The operations could be database operations like reading and writing to a disk, or mathematical and other operations like:
 - Searching
 - Sorting
 - Merging
 - Computation on field values

4. Communication Cost

- In most database systems the database resides in one station and various queries originate from different terminals. This will have impact on the performance of the system adding cost for query processing. Thus, the cost of transporting data between the database site and the terminal from where the query originate should be analyzed.

3. Query Execution Plans

- An execution plan for a relational algebra query consists of :
 - Relational algebra query tree
 - Information about the access methods to be used for each relation
 - The methods to be used in computing the relational operators stored in the tree.

1.9 Pipelining

- **Pipelined evaluation:** evaluate several operations simultaneously, passing the results of one operation on to the next.
- E.g., in expression tree, don't store result of
 - Instead, pass tuples directly to the join.. Similarly, don't store result of join, pass tuples directly to projection.
- Much cheaper than materialization: no need to store a temporary relation to disk.
- Pipelining may not always be possible – e.g., sort, hash-join.
- For pipelining to be effective, use evaluation algorithms that generate output tuples even as tuples are received for inputs to the operation.
- Pipelines can be executed in two ways: **demand driven** and **producer driven**
- In **demand driven** or **lazy** evaluation
 - system repeatedly requests next tuple from top level operation
 - Each operation requests next tuple from children operations as required, in order to output its next tuple
 - In between calls, operation has to maintain “**state**” so it knows what to return next
- In **producer-driven** or **eager** pipelining
 - Operators produce tuples eagerly and pass them up to their parents

- Buffer maintained between operators, child puts tuples in buffer, parent removes tuples from buffer
- if buffer is full, child waits till there is space in the buffer, and then generates more tuples
- System schedules operations that have space in output buffer and can process more input tuples
- Alternative name: **pull** and **push** models of pipelining
- Implementation of demand-driven pipelining
 - Each operation is implemented as an **iterator** implementing the following operations
 - open()
 - E.g. file scan: initialize file scan
 - state: pointer to beginning of file
 - E.g. merge join: sort relations;
 - state: pointers to beginning of sorted relations
 - next()
 - E.g. for file scan: Output next tuple, and advance and store file pointer
 - E.g. for merge join: continue with merge from earlier state till next output tuple is found. Save pointers as iterator state.
 - close()

Evaluation Algorithms for Pipelining

- Some algorithms are not able to output results even as they get input tuples
 - E.g. merge join, or hash join
 - intermediate results written to disk and then read back
- Algorithm variants to generate (at least some) results on the fly, as input tuples are read in

- E.g. hybrid hash join generates output tuples even as probe relation tuples in the in-memory partition (partition 0) are read in
- **Pipelined join technique:** Hybrid hash join, modified to buffer partition 0 tuples of both relations in-memory, reading them as they become available, and output results of any matches between partition 0 tuples
 - When a new r_0 tuple is found, match it with existing s_0 tuples, output matches, and save it in r_0
 - Symmetrically for s_0 tuples

Chapter 2: Database Security and Authorization

2.1 Data Integrity

- Security vs Integrity
 - Database security makes sure that the user is authorised to access information
 - Database integrity makes sure that (authorised) users use that information correctly
- Integrity constraints
 - Domain constraints apply to data types
 - Attribute constraints apply to columns
 - Relation constraints apply to rows in a single table
 - Database constraints apply between tables
- Intrarecord integrity (enforcing constraints on contents of fields, etc.)
- Referential Integrity (enforcing the validity of references between records in the database)
- Concurrency control (ensuring the validity of database updates in a shared multiuser environment)
- The constraints we wish to impose in order to protect the database from becoming inconsistent.
- Five types
 - i. Required data
 - ii. attribute domain constraints
 - iii. entity integrity
 - iv. referential integrity
 - v. enterprise constraints
- i. **Required Data**
 - Some attributes must always contain a value -- they cannot have a NULL value
 - For example:

- Every employee must have a job title.
- Every diveshop diveitem must have an order number and an item number

ii. Attribute domain constraints

- Every attribute has a *domain*, that is a set of values that are legal for it to use
- For example:
 - The domain of *sex* in the employee relation is “M” or “F”
- Domain ranges can be used to validate input to the database

iii. Entity integrity

- The primary key of any entity:
 - Must be Unique
 - Cannot be NULL

iv. Referential Integrity

- A “**foreign key**” links each occurrence in a relation representing a child entity to the occurrence of the parent entity containing the matching candidate (usually primary) key
- Referential Integrity means that if the foreign key contains a value, that value must refer to an existing occurrence in the parent entity
- For example:
 - Since the Order ID in the diveitem relation refers to a particular diveords item, that item must exist for referential integrity to be satisfied.
- Referential integrity options are declared when tables are defined (in most systems)
- There are many issues having to do with how particular referential integrity constraints are to be implemented to deal with insertions and deletions of data from the parent and child tables.

Insertion rules

- A row should not be inserted in the referencing (child) table unless there already exists a matching entry in the referenced table
- Inserting into the parent table should not cause referential integrity problems

- Sometimes a special NULL value may be used to create child entries without a parent or with a “dummy” parent

Deletion rules

- A row should not be deleted from the referenced table (parent) if there are matching rows in the referencing table (child)
- Three ways to handle this
 - Restrict -- disallow the delete
 - Nullify -- reset the foreign keys in the child to some NULL or dummy value
 - Cascade -- Delete all rows in the child where there is a foreign key matching the key in the parent row being deleted

v. Enterprise Constraints

- These are business rule that may affect the database and the data in it
 - for example, if a manager is only permitted to manage 10 employees then it would violate an enterprise constraint to manage more

Data and Domain Integrity

- This is now increasingly handled by the database. In Oracle, for example, when defining a table you can specify:

- CREATE TABLE table-name (

attr2 attr-type NOT NULL, *forbids NULL values*

attrN attr-type CHECK (attrN = UPPER(attrN)) *verifies that the data meets certain criteria*

attrO attr-type DEFAULT default_value); *Supplies default values*

Referential Integrity

- Ensures that dependent relationships in the data are maintained. In Oracle, for example:

- CREATE TABLE table-name (

attr1 attr-type PRIMARY KEY,

attr2 attr-type NOT NULL,

..., attrM attr-type REFERENCES owner.tablename(attrname) ON DELETE CASCADE, ...

Concurrency Control

- The goal is to support access by multiple users to the same data, at the same time
- It must assure that the transactions are *serializable* and that they are *isolated*
- It is intended to handle several problems in an uncontrolled system
- Specifically:
 - Lost updates
 - Inconsistent data states during access
 - Uncompleted (or committed) changes to data

No Concurrency Control: Lost updates

John

- Read account balance (balance = \$1000)
- Withdraw \$200 (balance = \$800)
- Write account balance (balance = \$800)

Marsha

- Read account balance (balance = \$1000)
- Withdraw \$300 (balance = \$700)
- Write account balance (balance = \$700)

Concurrency Control: Locking

- Locking levels
 - Database
 - Table
 - Block or page
 - Record
 - Field
- Types
 - Shared (S locks)
 - Exclusive (X locks)

Concurrency Control: Updates with X locking

John

- Lock account balance
- Read account balance (balance = \$1000)
- Withdraw \$200 (balance = \$800)
- Write account balance (balance = \$800)
- Unlock account balance

Marsha

- Read account balance (DENIED)
- Lock account balance
- Read account balance (balance = \$800)
- etc...

Concurrency Control: Deadlocks

John

- Place S lock
- Read account balance (balance = \$1000)
- Request X lock (denied)
- wait ...

Marsha

- Place S lock
- Read account balance (balance = \$1000)
- Request X lock (denied)
- wait. **Deadlock!**

- Avoiding deadlocks by maintaining tables of potential deadlocks and “backing out” one side of a conflicting transaction

Transaction Control in ORACLE

- Transactions are sequences of SQL statements that ORACLE treats as a unit
 - From the user’s point of view a private copy of the database is created for the duration of the transaction
- Transactions are started with SET TRANSACTION, followed by the SQL statements
- Any changes made by the SQL are made permanent by COMMIT

- Part or all of a transaction can be undone using ROLLBACK
- COMMIT;
- SET TRANSACTION READ ONLY;
- SELECT NAME, ADDRESS FROM WORKERS;
- SELECT MANAGER, ADDRESS FROM PLACES;
- COMMIT;
- *Freezes the data for the user in both tables before either select retrieves any rows, so that changes that occur concurrently will not show up*
- *Commits before and after ensure any uncompleted transactions are finish, and then release the frozen data when done*
- Savepoints are places in a transaction that you may ROLLBACK to (called checkpoints in other DBMS)
 - SET TRANSACTION...;
 - SAVEPOINT ALPHA;
 - *SQL STATEMENTS...*
 - IF (CONDITION) THEN ROLLBACK TO SAVEPOINT ALPHA;
 - SAVEPOINT BETA;
 - *SQL STATEMENTS...*
 - IF ...;
 - COMMIT;

2.2 Database Security

- Views or restricted subschemas
- Authorization rules to identify users and the actions they can perform
- User-defined procedures (and rule systems) to define additional constraints or limitations in using the database
- Encryption to encode sensitive data

- Authentication schemes to positively identify a person attempting to gain access to the database
- Database security is about controlling access to information
 - Some information should be available freely
 - Other information should only be available to certain people or groups
- Many aspects to consider for security
 - Legal issues
 - Physical security
 - OS/Network security
 - Security policies and protocols
 - Encryption and passwords
 - DBMS security

2.2.1 DBMS Security Support

- DBMS can provide some security
 - Each user has an account, username and password
 - These are used to identify a user and control their access to information
- DBMS verifies password and checks a user's permissions when they try to
 - Retrieve data
 - Modify data
 - Modify the database structure

2.2.2 Permissions and Privilege

- SQL uses privileges to control access to tables and other database objects
 - SELECT privilege
 - INSERT privilege
 - UPDATE privilege
 - DELETE privilege

- The owner (creator) of a database has all privileges on all objects in the database, and can grant these to others
- The owner (creator) of an object has all privileges on that object and can pass them on to others

2.2.3 Authorization Rules

- Most current DBMS permit the DBA to define “access permissions” on a table by table basis (at least) using the GRANT and REVOKE SQL commands
- Some systems permit finer grained authorization (most use GRANT and REVOKE on variant views)

2.2.4 Privileges in SQL

GRANT <privileges>

ON <object>

TO <users>

[WITH GRANT OPTION]

- **<privileges>** is a list of

SELECT <columns>, INSERT <columns>, DELETE, and UPDATE <columns>, or simply ALL

- **<users>** is a list of user names or **PUBLIC**
- **<object>** is the name of a table or view
- **WITH GRANT OPTION** means that the users can pass their privileges on to others

Privileges Examples

GRANT ALL ON Employee

TO Manager

WITH GRANT OPTION

The user ‘Manager’ can do anything to the Employee table, and can allow other users to do the same (by using **GRANT** statements)

GRANT SELECT,

UPDATE(Salary) ON

Employee TO Finance

The user 'Finance' can view the entire Employee table, and can change Salary values, but cannot change other values or pass on their privilege

Removing Privileges

- If you want to remove a privilege you have granted you use

REVOKE <privileges>

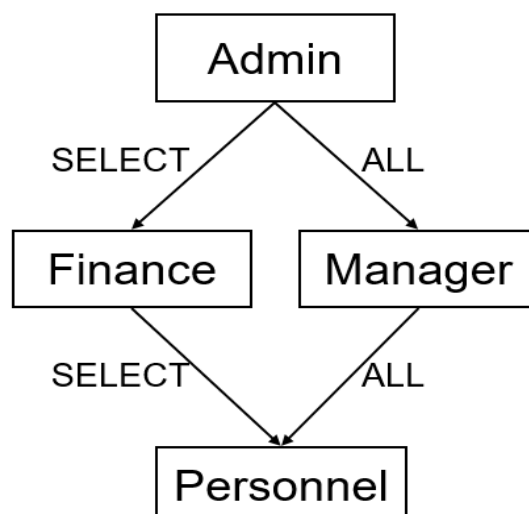
ON <object>

FROM <users>

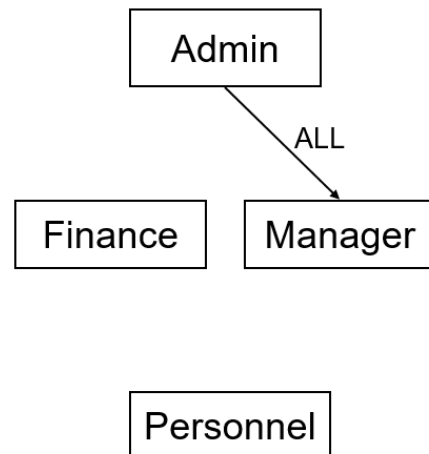
- If a user has the same privilege from other users then they keep it
- All privileges dependent on the revoked one are also revoked

Example

- 'Admin' grants ALL privileges to 'Manager', and SELECT to 'Finance' with grant option
- 'Manager' grants ALL to Personnel
- 'Finance' grants SELECT to Personnel



- 'Manager' revokes ALL from 'Personnel'
 - 'Personnel' still has SELECT privileges from 'Finance'
- 'Admin' revokes SELECT from 'Finance'
 - Personnel loses SELECT also



Views

- A subset of the database presented to some set of users
 - SQL:

CREATE VIEW viewname AS SELECT field1, field2, field3,..., FROM table1, table2
WHERE <where clause>;

- Note: “queries” in Access function as views

Restricted Views

- Main relation has the form:

Name	C_name	Dept	C_dept	Prof	C_prof	TC
J Smith	S	Dept1	S	Cryptography	TS	TS
M Doe	U	Dept2	S	IT Security	S	S
R Jones	U	Dept3	U	Secretary	U	U

U = unclassified: S = Secret: TS = Top Secret

S-view of the data

NAME	Dept	Prof
J Smith	Dept1	---
M Doe	Dept2	IT Security
R Jones	Dept3	Secretary

U-view of the data

NAME	Dept	Prof
M Doe	---	---
R Jones	Dept3	Secretary

- Privileges work at the level of tables
 - You can restrict access by column
 - You cannot restrict access by row
- Views, along with privileges, allow for customised access
- Views provide ‘derived’ tables
 - A view is the result of a SELECT statement which is treated like a table
 - You can SELECT from (and sometimes UPDATE etc) views just like tables

Creating Views

CREATE VIEW <name>

AS <select stmt>

- <name> is the name of the new view

- **<select stmt>** is a query that returns the rows and columns of the view
- Example
 - We want each user to be able to view the names and phone numbers (only) of those employees in their own department
- Example
 - We want each user to be able to view the names and phone numbers (only) of those employees in their own department
 - In Oracle, you can refer to the current user as **USER**

Employee				
ID	Name	Phone	Department	Salary
E158	Mark	x6387	Accounts	£15,000
E159	Mary	x6387	Marketing	£15,000
E160	Jane	x6387	Marketing	£15,000

View example

```
CREATE VIEW OwnDept AS
```

```
SELECT Name, Phone FROM Employee
```

```
WHERE Department =
```

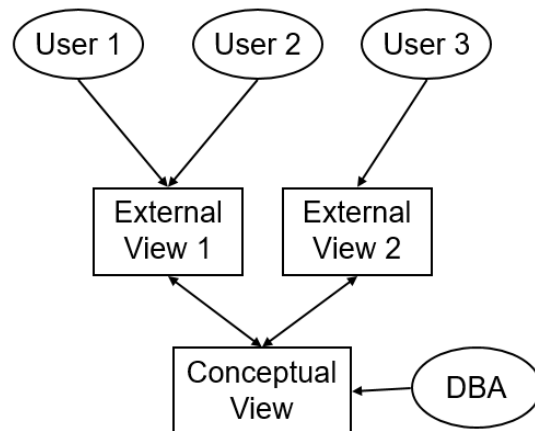
```
(SELECT Department FROM Employee
```

```
WHERE name = USER)
```

```
GRANT SELECT ON OwnDept TO PUBLIC
```

Using Views and Privileges

- Views and privileges are used together to control access
 - A view is made which contains the information needed
 - Privileges are granted to that view, rather than the underlying tables



View Updating

- Views are like virtual tables
 - Their value depends on the 'base' tables that they are defined from
 - You can select from views just like a table
 - What about update, insert, and delete?
- Updating views
 - Updates to the base tables change the views and vice-versa
 - It is often not clear how to change the base tables to make the desired change to the view
- For a view to be updatable, the defining query of the view should satisfy certain conditions:
 - Every element in SELECT is a column name
 - Should not use DISTINCT
 - View should be defined on a single table (no join, union, etc. used in FROM)
 - WHERE should not have nested SELECTs
 - Should not use GROUP BY or HAVING

Using Views and Privileges

To restrict someone's access to a table

- Create a view of that table that shows only the information they need to see

- Grant them privileges on the view
- Revoke any privileges they have on the original table employee
- We want to let the user 'John' read the department and name, and be able to update the department (only)
- Create a view

CREATE VIEW forJohn

AS SELECT Name,

Department

FROM Employee

Set the privileges

GRANT SELECT,

UPDATE (Department)

ON forJohn

TO John

REVOKE ALL ON Employee FROM John

2.3 Backup and Recovery

- Backup
- Journaling (audit trail)
- Checkpoint facility
- Recovery manager

Disaster Recovery Planning



From Toigo "Disaster Recovery Planning"

Threats to Assets and Functions

- Water
- Fire
- Power Failure
- Mechanical breakdown or software failure
- Accidental or deliberate destruction of hardware or software
 - By hackers, disgruntled employees, industrial saboteurs , terrorists, or others

Kinds of Records

- Class I: VITAL
 - Essential, irreplaceable or necessary to recovery
- Class II: IMPORTANT
 - Essential or important, but reproducible with difficulty or at extra expense
- Class III: USEFUL

- Records whose loss would be inconvenient, but which are replaceable
- Class IV: NONESSENTIAL
 - Records which upon examination are found to be no longer necessary

Offsite Storage of Data

- Early offsite storage facilities were often intended to survive atomic explosions
- PRISM International directory
 - <http://www.prismintl.org/>
- Mirror sites (Hot sites)

Chapter 3: Transaction Processing Concepts

3.1 Introduction to Transaction

- **A Transaction:**
 - Logical unit of database processing that includes one or more access operations (read -retrieval, write - insert or update, delete).
- Examples include ATM transactions, credit card approvals, flight reservations, hotel check-in, phone calls, supermarket scanning, academic registration and billing.
- **Transaction boundaries:**
 - Any single transaction in an application program is bounded with *Begin* and *End* statements.
- An **application program** may contain several transactions separated by the *Begin* and *End* transaction boundaries.

SIMPLE MODEL OF A DATABASE:

- **A database** is a collection of named data items
- **Granularity** of data - a field, a record , or a whole disk block that measure the size of the data item
- Basic operations that a transaction can perform are **read** and **write**
 - *read_item(X)*: Reads a database item named X into a program variable. To simplify our notation, we assume that the program variable is also named X.
 - *write_item(X)*: Writes the value of program variable X into the database item named X.
- Basic unit of data transfer from the disk to the computer main memory is one block.
- *read_item(X)* command includes the following steps:
 - Find the address of the disk block that contains item X.
 - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
 - Copy item X from the buffer to the program variable named X.
- **write_item(X)** command includes the following steps:

- Find the address of the disk block that contains item X.
- Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
- Copy item X from the program variable named X into its correct location in the buffer.
- Store the updated block from the buffer back to disk (either immediately or at some later point in time).
- The DBMS maintains a number of buffers in the main memory that holds database disk blocks which contains the database items being processed.
 - When this buffers are occupied and
 - if there is a need for additional database block to be copied to the main memory ;
- Some buffer management policy is used to choose for replacement but if the chosen buffer has been modified, it must be written back to disk before it is used.
- *Two sample transactions:*
 - (a) Transaction T1
 - (b) Transaction T2

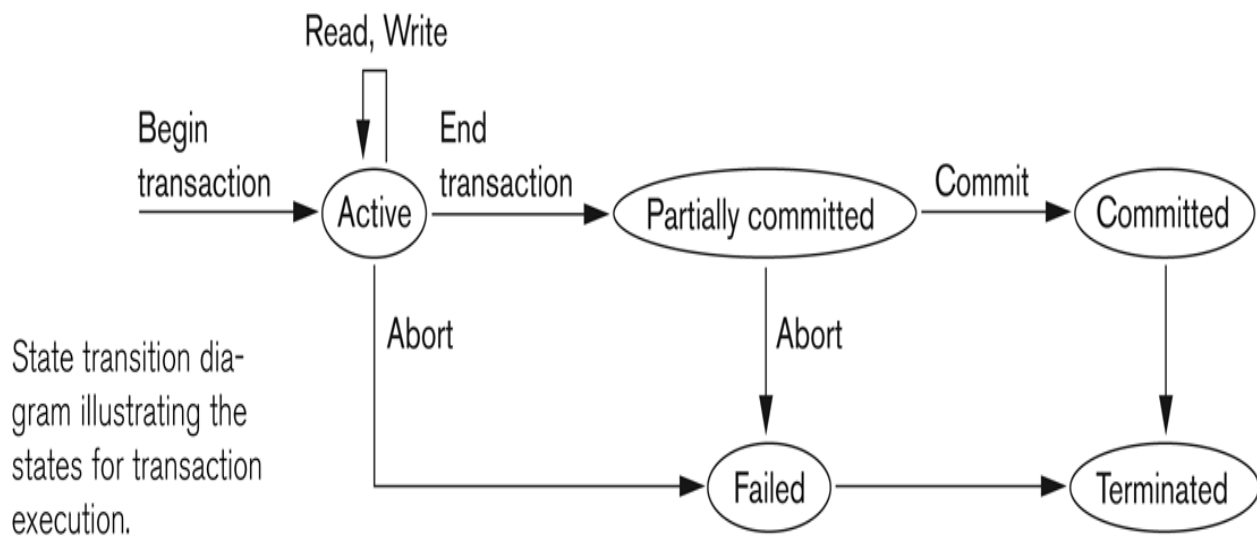
(a)	T_1	(b)	T_2
	<code>read_item (X);</code> <code>X:=X-N;</code> <code>write_item (X);</code> <code>read_item (Y);</code> <code>Y:= Y+N;</code> <code>write_item (Y);</code>		<code>read_item (X);</code> <code>X:=X+M;</code> <code>write_item (X);</code>

3.2 Transaction and System Concepts

- A **transaction** is an atomic unit of work that is either completed in its entirety or not done at all.
 - For recovery purposes, the system needs to keep track of when the transaction starts, terminates, and commits or aborts.
- Transaction states:
 - Active state -indicates the beginning of a transaction execution

- Partially committed state shows the end of read/write operation but this will not ensure permanent modification on the data base
- Committed state -ensures that all the changes done on a record by a transition were done persistently
- Failed state happens when a transaction is aborted during its active state or if one of the rechecking is fails
- Terminated State -corresponds to the transaction leaving the system

State transition diagram illustrating the states for transaction execution



- T in the following discussion refers to a unique **transaction-id** that is generated automatically by the system and is used to identify each transaction:
- Types of log record:
 - [start_transaction,T]: Records that transaction T has started execution.
 - [write_item,T,X,old_value,new_value]: Records that transaction T has changed the value of database item X from old_value to new_value.
 - [read_item,T,X]: Records that transaction T has read the value of database item X.
 - [commit,T]: Records that transaction T has completed successfully, and affirms that its effect can be committed (recorded permanently) to the database.
 - [abort,T]: Records that transaction T has been aborted.

Desirable Properties of Transactions

To ensure data integrity, DBMS should maintain the following ACID properties:

- **Atomicity:** A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all.
- **Consistency preservation:** A correct execution of the transaction must take the database from one consistent state to another.
- **Isolation:** A transaction should not make its updates visible to other transactions until it is committed; this property, when enforced strictly, solves the temporary update problem and makes cascading rollbacks of transactions unnecessary
- **Durability or permanency:** Once a transaction changes the database and the changes are committed, these changes must never be lost because of subsequent failure.

Example

- Suppose that T_i is a transaction that transfer 200 birr from account CA2090(which is 5,000 Birr) to SB2359(which is 3,500 birr) as follows
 - ***Read(CA2090)***
 - ***CA2090 = CA2090 - 200***
 - ***Write(CA2090)***
 - ***Read(SB2359)***
 - ***SB2359 = SB2359 + 200***
 - ***Write(SB2359)***
- Atomicity- either all or none of the above operation will be done – this is materialized by **transaction management** component of DBMS
- Consistency-the sum of CA2090 and SB2359 be unchanged by the execution of T_i i.e 8500- this is the responsibility of application programmer who codes the transaction
- Isolation- when several transaction are being processed concurrently on a data item they may create many inconsistent problems. So handling such case is the responsibility of Concurrency control component of the DBMS
- Durability - once T_i writes its update this will remain there when the database restarted from failure . This is the responsibility of **recovery management** components of the DBMS

3.3 Transaction Processing

- **Single-User System:**

- *At most one user at a time can use the database management system.*
- *Eg. Personal computer system*

- **Multiuser System:**

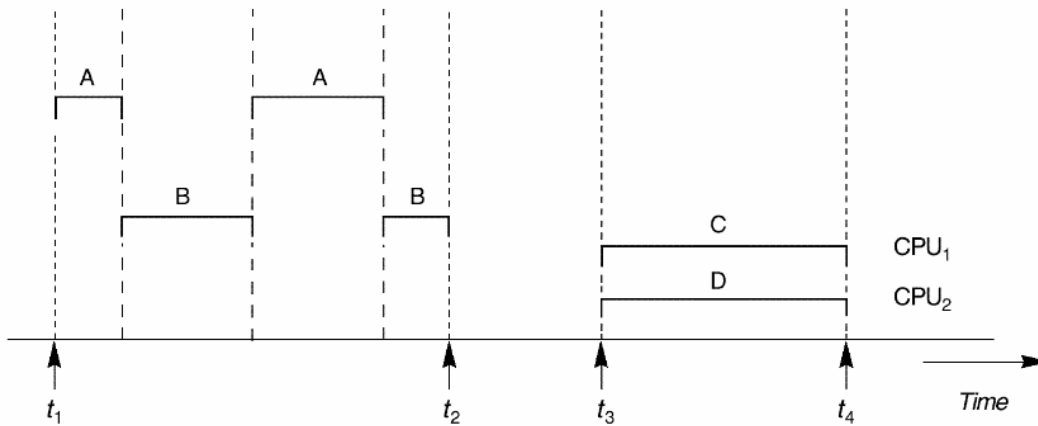
- *Many users can access the DBMS concurrently.*
- *Eg. Air line reservation, Bank and the like system are operated by many users who submit transaction concurrently to the system*
- *This is achieved by multiprogramming, which allows the computer to execute multiple programs /processes at the same time.*

- **Concurrency**

- ***Interleaved processing:***
 - Concurrent execution of processes is interleaved in a single CPU using for example, round robin algorithm

Advantages:

- keeps the CPU busy when the process requires I/O by switching to execute another process rather than remaining idle during I/O time and hence this will increase system throughput (average no. of transactions completed within a given time)
- prevents long process from delaying other processes (minimize unpredictable delay in the response time).
- ***Parallel processing:***
 - If Processes are concurrently executed in multiple CPUs.



Interleaved processing versus parallel processing of concurrent transactions.

Problems of Concurrent Sharing

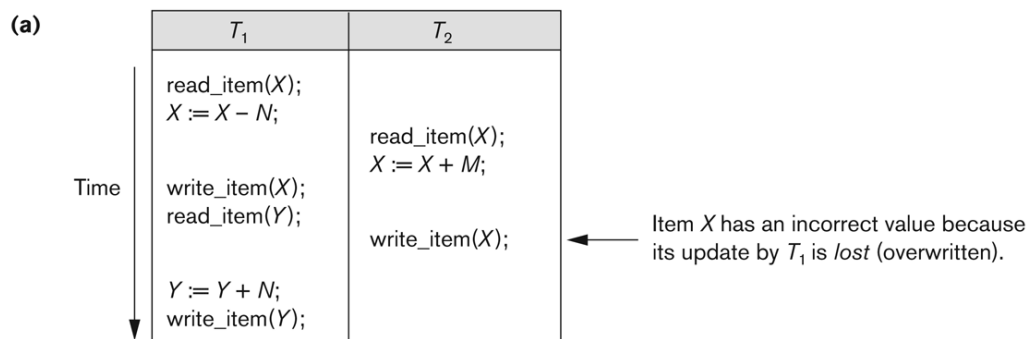
Why Concurrency Control is needed: Three cases

i. *The Lost Update Problem*

- This occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database item incorrect.

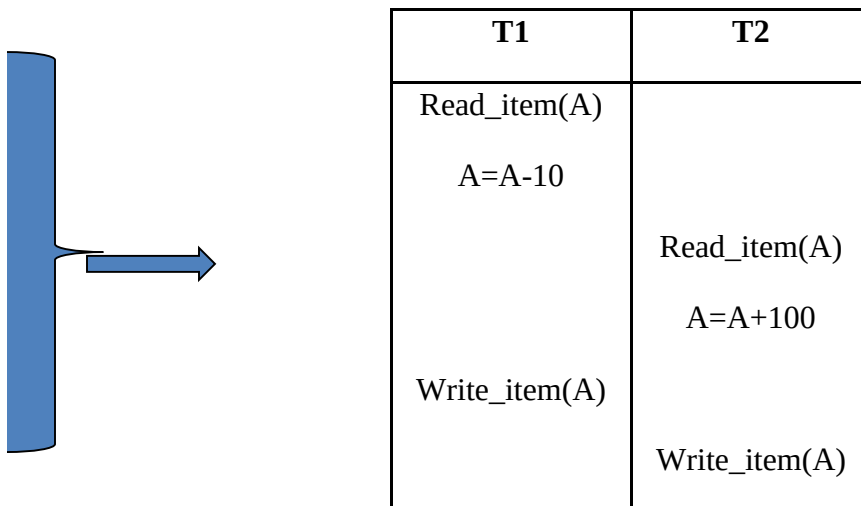
Figure 1.3

Some problems that occur when concurrent execution is uncontrolled.



- E.g. Account with balance A=100.
 - T1 reads the account A

- T1 withdraws 10 from A
- T1 makes the update in the Database
- T2 reads the account A
- T2 adds 100 on A
- T2 makes the update in the Database
- In the above case, if done one after the other (serially) then we have no problem.
 - If the execution is T1 followed by T2 then A=190
 - If the execution is T2 followed by T1 then A=190
- But if they start at the same time in the following sequence:
- T1 reads the account A=100
- T1 withdraws 10 making the balance A=90
- T2 reads the account A=100
- T2 adds 100 making A=200
- T1 makes the update in the Database A=90
- T2 makes the update in the Database A=200
- After the successful completion of the operation the final value of A will be 200 which override the update made by the first transaction that changed the value from 100 to 90.



ii. ***The Temporary Update (or Dirty Read) Problem***

- This occurs when one transaction updates a database item and then the transaction fails for some reason .
- The updated item is accessed by another transaction before it is changed back to its original value. Based on the above scenario:

iii. ***The Incorrect Summary Problem***

- If one transaction is calculating an aggregate summary function on a number of records while other transactions are updating some of these records, the aggregate function may calculate some values before they are updated and others after they are updated.

Example: T1 would like to add the values of A=10, B=20 and C=30. after the values are read by T1 and before its completion, T2 updates the value of B to be 50. at the end of the execution of the two transactions T1 will come up with the sum of 60 while it should be 90 since B is updated to 50

3.4 Concept of Schedules and Serializability

Characterizing Schedules

- **Transaction schedule or history:**
 - When transactions are executing concurrently in an interleaved fashion, the order of execution of operations from the various transactions forms what is known as a transaction schedule (or history).
- A **schedule** S of n transactions T1, T2, ..., Tn:
 - It is an ordering of the operations of the transactions subject to the constraint that, for each transaction Ti that participates in S, the operations of T1 in S must appear in the same order in which they occur in Ti.
 - Note, however, that operations from other transactions Tj can be interleaved with the operations of Ti in S. Eg. Consider the following example:

S_a: r2(X);w2(X);r1(X);w1(X);a2;

- Two operations in a schedule are said to be conflict if they satisfy the following conditions.
 - They belongs to different transaction
 - They access the same data item X

- At least one of the operation is a write_Item(X)

Eg. Sa: r1(X); r2(x); w1(X); r1(Y); W2(X); W1(Y);

- r1(X) and w2(X)
 - r2(X) and w1(X);
 - W1(X) and w2(X)
- } Conflicting operations

- r1(X) and r2(X)
 - W2(X) and w1(Y)
 - R1(x) and w1(x)
- } No Conflict, why?

i. Schedules classified by recoverability:

- **Recoverable schedule:**

- One where no committed transaction needs to be rolled back.
- A schedule S is recoverable if no transaction Tj in S commits until all transactions Ti that have written an item that Tj reads have committed. Examples,

- **Sc:** r1(X); w1(X); r2(X); r1(Y); w2(x); c2; a1; not recoverable

- **Sd:** r1(X); w1(X); r2(X); r1(Y); w2(X); w1(Y); c1; c2;

- **Se:** r1(X); w1(X); r2(X); r1(Y); w2(x); w1(Y); a1; a2;

} Recoverable

- **Cascadeless schedule:**

- One where every transaction reads only the items that are written by committed transactions. Eg.

- **Sf:** r1(X); w1(X); r1(Y); c1; r2(X); w2(X); w1(Y); c2;

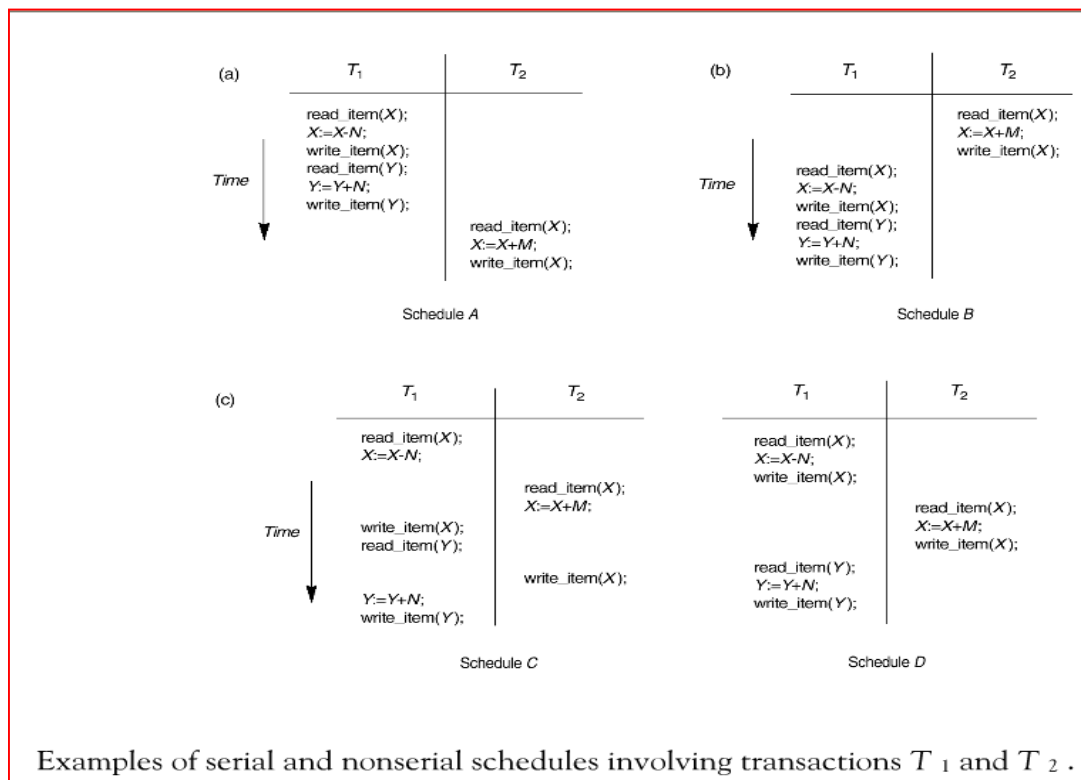
- **Strict Schedules:**

- A schedule in which a transaction can neither read or write an item X until the last transaction that wrote X has committed/aborted.

- Eg. **Sg:** w1(X,5); c1; w2(x,8);

ii. Characterizing Schedules based on Serializability

- The concept of Serializable of schedule is used to identify which schedules are correct when concurrent transactions executions have interleaving of their operations in the schedule.
- Serial schedule:
 - A schedule S is serial if, for every transaction T_i participating in the schedule, all the operations of T_i are executed consecutively in the schedule. Otherwise, the schedule is called nonserial schedule.
 - *For example*, in the banking example suppose there are two transaction where one transaction calculate the interest on the account and another deposit some money into the account. hence the order of execution is important for the final result.
- Serializable schedule:
 - a schedule whose effect on any consistent database instance is identical to that of some complete serial schedule over the set of *committed* transactions in S .
 - A nonserial schedule S is serializable, is equivalent to say that it is correct to the result of one of the serial schedule .Example,



- Result equivalent:
 - Two schedules are called result equivalent if they produce the same final state of the database
 - Two types of equivalent schedule: Conflict and view

i. Conflict equivalent:

- Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules. Eg
 - S1: r1(x); w2(x) & S2: w2(x); r1(x)
 - S1: w1(x); w2(x); & S2: w2(x); w1(x);
- Conflict serializable:
- A schedule S is said to be *conflict serializable* if it is conflict equivalent to some serial schedule S'.
- Every conflict serializable schedule is serializable .

} Not conflict equivalent

- If you can transform an interleaved schedule by swapping consecutive non-conflicting operations of different transactions into a serial schedule, then the original schedule is *conflict serializable*.

ii. Two schedules are said to be view equivalent if the following three conditions hold:

1. The same set of transactions participates in S and S', and S and S' include the same operations of those transactions.
2. If Ti reads a value A written by Tj in S1, it must also read the value of A written by Tj in S2
3. for each data object A, the transaction that perform the final write on A in S1 must also perform the final write on A in S2

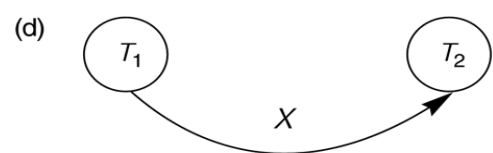
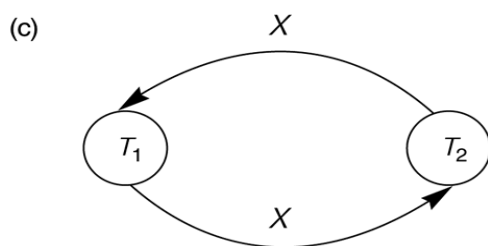
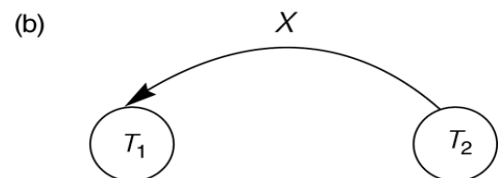
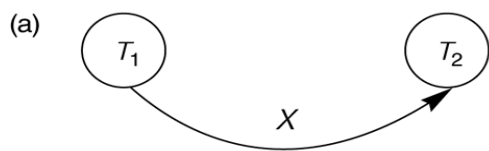
Testing for conflict serializable Algorithm

- Looks at only read_Item (X) & write_Item (X) operations
- Constructs a precedence graph (serialization graph) - a graph with directed edges

- An edge is created from T_i to T_j if one of the operations in T_i appears before a conflicting operation in T_j
- The schedule is serializable if and only if the precedence graph has no cycles.

Constructing the Precedence Graphs

- Example: Constructing the precedence graphs for schedules A to D from (Slide No. 23) to test for conflict serializability.
 - (a) Precedence graph for serial schedule A.
 - (b) Precedence graph for serial schedule B.
 - (c) Precedence graph for schedule C (not serializable).
 - (d) Precedence graph for schedule D (serializable, equivalent to schedule A).



Another example of serializability Testing

Another example of serializability testing.
 (a) The read and write operations of three transactions T_1 , T_2 , and T_3 . (b) Schedule E. (c) Schedule F.

(a)

Transaction T_1	Transaction T_2	Transaction T_3
read_item(X);	read_item(Z);	read_item(Y);
write_item(X);	read_item(Y);	read_item(Z);
read_item(Y);	write_item(Y);	write_item(Y);
write_item(Y);	read_item(X);	write_item(Z);
	write_item(X);	

Another example of serializability testing.
 (a) The read and write operations of three transactions T_1 , T_2 , and T_3 . (b) Schedule E. (c) Schedule F.

(b)

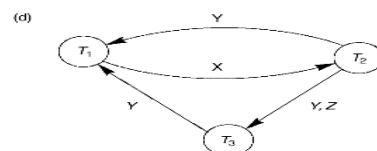
Transaction T_1	Transaction T_2	Transaction T_3
read_item(X); write_item(X);	read_item(Z); read_item(Y); write_item(Y);	read_item(Y); read_item(Z);
read_item(Y); write_item(Y);	read_item(X); write_item(X);	write_item(Y); write_item(Z);

Schedule E

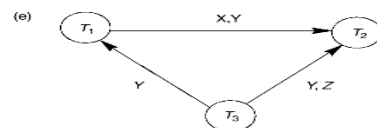
Another example of serializability testing.
 (a) The read and write operations of three transactions T_1 , T_2 , and T_3 . (b) Schedule F.

(c)

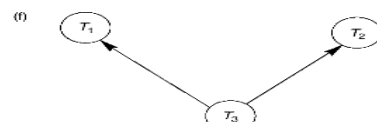
Transaction T_1	Transaction T_2	Transaction T_3
read_item(X); write_item(X);		read_item(Y); read_item(Z); write_item(Y);



Equivalent serial schedules
 None
 Reason
 cycle $X(T_1 \rightarrow T_2), Y(T_2 \rightarrow T_1)$
 cycle $X(T_1 \rightarrow T_2), YZ(T_2 \rightarrow T_3), Y(T_3 \rightarrow T_1)$



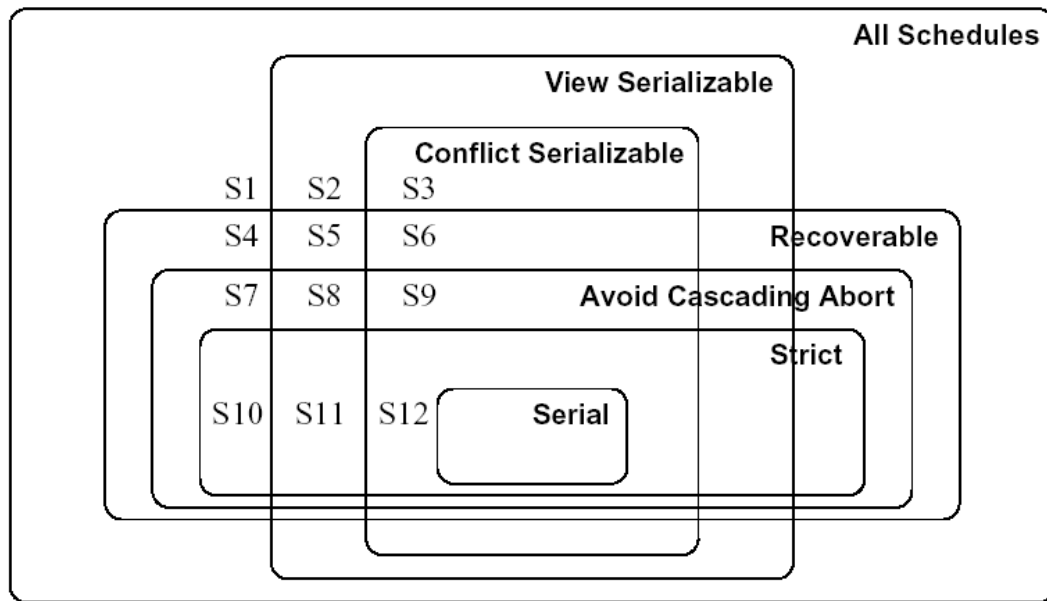
Equivalent serial schedules
 $T_3 \rightarrow T_1 \rightarrow T_2$



Equivalent serial schedules
 $T_3 \rightarrow T_1 \rightarrow T_2$
 $T_3 \rightarrow T_2 \rightarrow T_1$

Another example of serializability testing. (d) Precedence graph for schedule E. (e) Precedence graph for schedule F. (f) Precedence graph with two equivalent serial schedules.

Summary of Schedule types



Venn Diagram for Classes of Schedules

3.5 Transaction Support in SQL

- A **single** SQL statement is always considered to be **atomic**.
 - Either the statement completes execution without error or it fails and leaves the database unchanged.
- Every transaction has three characteristics: Access mode, Diagnostic size and isolation
 - **Access mode:**
 - READ ONLY or READ WRITE
 - If the access mode is Read ONLY , INSERT, DELET , UPDATE & CREATE commands cannot be executed on the data base
 - The default is READ WRITE unless the isolation level of READ UNCOMMITTED is specified, in which case READ ONLY is assumed.
 - **Diagnostic size n**, specifies an integer value n, indicating the number of error conditions that can be held simultaneously in the diagnostic area.

- **Isolation level** can be
 - READ UNCOMMITTED,
 - READ COMMITTED,
 - REPEATABLE READ or
 - SERIALIZABLE. The default is SERIALIZABLE.

SET TRANSACTION

READ WRITE | READ ONLY

ISOLATION LEVEL REPEATABLE READ| READ UNCOMMITTED|
SERIALIZABLE;

- BEGIN TRANSACTION statement and end with the COMMIT or ROLLBACK statement.
- SQL transaction Examples:

Example 1: Commit Transaction

BEGIN TRANSACTION;

DELETE FROM HumanResources.JobCandidate

WHERE JobCandidateID = 13;

COMMIT;

Example 2: Rollback Transaction

CREATE TABLE ValueTable (id int);

BEGIN TRANSACTION;

INSERT INTO ValueTable VALUES(1);

INSERT INTO ValueTable VALUES(2);

ROLLBACK;

- With SERIALIZABLE: the interleaved execution of transactions will adhere to the notion of serializability.
- However, if any transaction executes at a lower level, then serializability may be violated.

Potential problem with lower isolation levels: Four types

i. Unrepeatable Reads: RW Conflicts

- a transaction $T2$ could change the value of an object A that has been read by a transaction $T1$, while $T1$ is still in progress.
- If $T1$ tries to read the value a again it will get a different value

T1:	R(A),	R(A), W(A), C
T2:	R(A), W(A), C	

ii. Reading Uncommitted Data (“dirty reads”): WR Conflicts

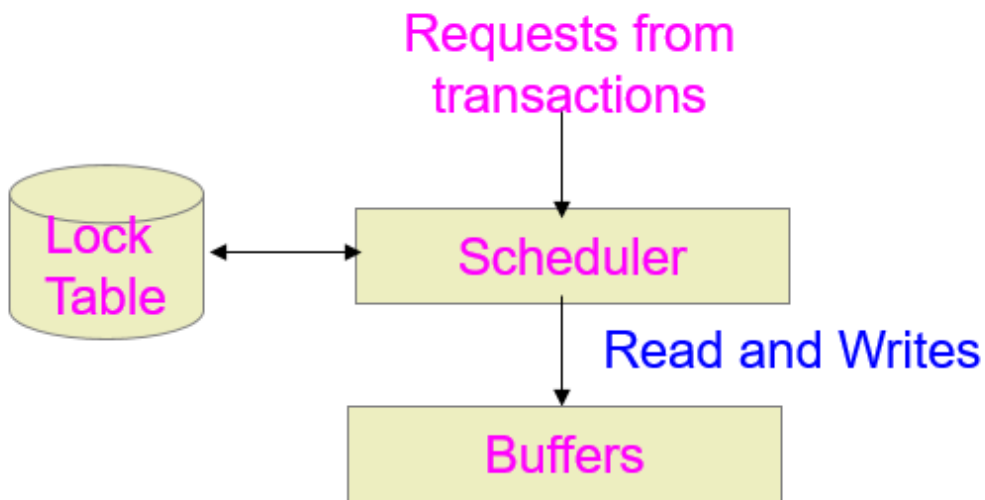
- a transaction $T2$ could read a database object A that has been modified by another transaction $T1$, which has not yet committed.

T1:	R(A), W(A),	R(B), W(B), Abort
T2:	R(A), W(A), C	

Chapter 4: Concurrency Controlling Techniques

4.1 Database Concurrency Control

- **Transaction Processor is divided into:**
 - A concurrency-control manager, or scheduler, responsible for assuring isolation of transactions
 - A logging and recovery manager, responsible for the durability of transactions.
- The scheduler (**concurrency-control manager**) must assure that the individual actions of multiple transactions are executed in such an order that the net effect is the same as if the transactions had in fact executed one-at-a-time.

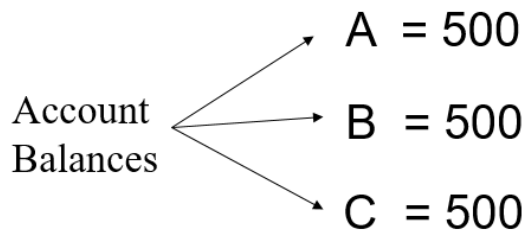


1. Purpose of Concurrency Control

- To enforce Isolation (through mutual exclusion) among conflicting transactions.
 - To preserve database consistency through consistency preserving execution of transactions.
 - To resolve read-write and write-write conflicts.
- A typical scheduler does its work by maintaining locks on certain pieces of the database. These locks prevent two transactions from accessing the same piece of data at the same time. Example:
 - In concurrent execution environment if T1 conflicts with T2 over a data item A, then the existing concurrency control decides if T1 or T2 should get the A and if the other transaction is rolled-back or waits.

Example:

Bank database: 3 Accounts



Property: $A + B + C = 1500$

Money does not leave the system

Example

Transaction T1: Transfer
100 from A to B

Read (A, t)

$t = t - 100$

Write (A, t)

Read (B, t)

$t = t + 100$

Write (B, t)

Transaction T2: Transfer
100 from A to C

Read (A, s)

$s = s - 100$

Write (A, s)

Read (C, s)

$s = s + 100$

Write (C, s)

Transaction T1	Transaction T2	A	B	C
Read (A, t)		500	500	500
t = t - 100				
	Read (A, s)			
	s = s - 100			
	Write (A, s)	400	500	500
Write (A, t)		400	500	500
Read (B, t)				
t = t + 100				
Write (B, t)		400	600	500
	Read (C, s)			
	s = s + 100			
	Write (C, s)	400	600	600

Schedule

$$400 + 600 + 600 = 1600$$

4.2 Concurrency Control Techniques

- **Basic concurrency control techniques:**
 - Locking,
 - Timestamping
 - Optimistic methods
- The First two are conservative approaches: delay transactions in case they conflict with other transactions.
- Optimistic methods assume conflict is rare and only check for conflicts at commit.

4.2.1 Locking

- Lock is a variable associated with a data item that describes the status of the data item with respect to the possible operations that can be applied to it.
- Generally, a transaction must claim a *shared (read)* or *exclusive (write)* lock on a data item before read or write.
- Lock prevents another transaction from modifying item or even reading it, in the case of a write lock.
- Locking is an operation which secures
 - (a) permission to Read
 - (b) permission to Write a data item for a transaction.
 - Example:
 - Lock (X). Data item X is locked in behalf of the requesting transaction.
- Unlocking is an operation which removes these permissions from the data item.
 - Example:
 - Unlock (X): Data item X is made available to all other transactions.
- Lock and Unlock are Atomic operations.
- Two locks modes:
 - (a) shared (read) (b) exclusive (write).
 - Shared mode: shared lock (X)
 - More than one transaction can apply share lock on X for reading its value but no write lock can be applied on X by any other transaction.
 - Exclusive mode: Write lock (X)
 - Only one write lock on X can exist at any time and no shared lock can be applied by any other transaction on X.
- Conflict matrix

	Read	Write
Read	Y	N
Write	N	N

- Lock

Manager:

- Managing locks on data items.

- Lock table:

- Lock manager uses it to store the identify of transaction locking a data item, the data item, lock mode and pointer to the next data item locked. One simple way to implement a lock table is through linked list.

- Database requires that all transactions should be well-formed. A transaction is well-formed if:

- It must lock the data item before it reads or writes to it.
- It must not lock an already locked data items and it must not try to unlock a free data item.

Locking - Basic Rules

- It has two operations : Lock_item(X) and unLock_item(X)
- A transaction request access to an item X by first issuing a lock_Item(x) operation .
- If lock (x)=1, the transaction is forced to wait.
- If lock (X)= 0; it is set to 1 and the transaction is allowed to access x
- When a transaction finished operation on X it issues an Unlock _item operation which set lock(x) to 0 so that X may be accessed by another transaction
- If transaction has shared lock on item, can read but not update item.
- If transaction has exclusive lock on item, can both read and update item.
- Reads cannot conflict, so more than one transaction can hold shared locks simultaneously on same item.
- Exclusive lock gives transaction exclusive access to that item.
- The following code performs the read operation: **read_lock(X):**

B: if LOCK (X) = “unlocked” then

begin

LOCK (X) ← “read-locked”;

no_of_reads (X) ← 1;

end

else if LOCK (X) ← “read-locked” then

no_of_reads (X) ← no_of_reads (X) +1

else begin

wait (until LOCK (X) = “unlocked” and

the lock manager wakes up the transaction);

go to B

end;

- The following code performs the write lock operation: **write_lock(X):**

B: if LOCK (X) = “unlocked” then

LOCK (X) ← “write-locked”;

else

wait (until LOCK(X) = “unlocked”

and the lock manager wakes up the transaction);

goto B

end;

- **Lock conversion**

- Lock upgrade: existing read lock to write lock

if T_i has a read-lock (X) and T_j has no read-lock (X) ($i \neq j$) then

convert read-lock (X) to write-lock (X)

else

force T_i to wait until T_j unlocks X

- Lock downgrade: existing write lock to read lock

T_i has a write-lock (X) (*no transaction can have any lock on X^*) convert write-lock (X) to read-lock (X)

- Using such locks in the transaction do not guarantee serializability of schedule on its own: example

Example : Incorrect Locking Schedule

Schedule:

Write Lock(X)

Read (X)

$X = X + 100$

Write(X)

Unlock(X)

Write Lock(X)

Read (X)

$X = X * 1.1$

Write(X)

Unlock(X)

Write Lock(Y)

Read (Y)

$Y = Y * 1.1$

Write(Y)

Unlock(Y)

Commit

write_lock(Y)

read(Y)

$Y = Y - 100$

write(Y)

unlock(Y)

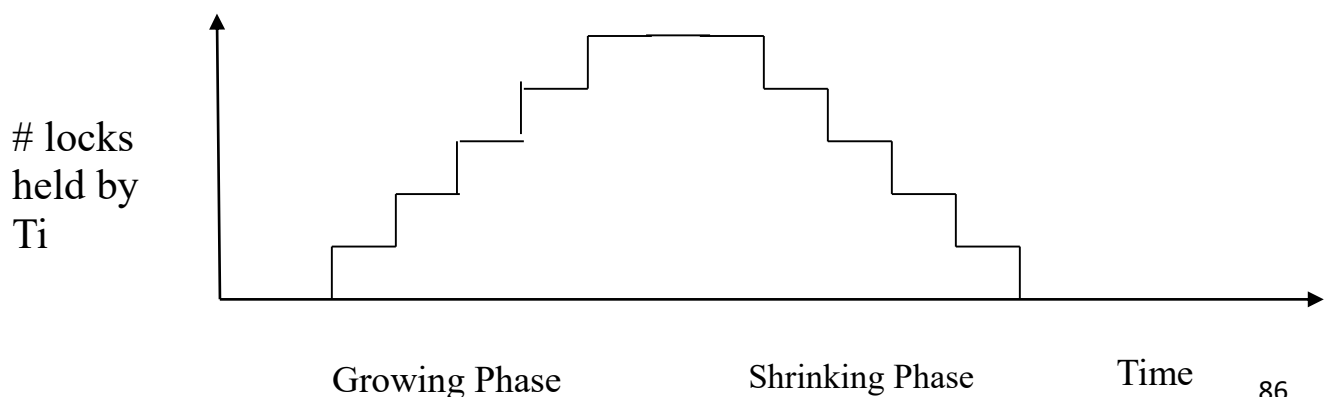
commit

- If at start, $X = 100$, $Y = 400$, result should be:
 - $X = 220$, $y = 330$, if T_1 executes before T_2 , or

- $X = 210, Y = 340$, if T_2 executes before T_1
- However, result gives $X = 220$ and $Y = 340$.
- S is not a serializable schedule. Why?
- Problem is that transactions release locks too soon, resulting in loss of total isolation and atomicity.
- To guarantee serializability, we need an additional protocol concerning the positioning of lock and unlock operations in every transaction.

4.2.2 Two-Phase Locking Techniques: The algorithm

- **Transaction follows 2PL protocol if all locking operations precede first unlock operation in the transaction.**
- Every transaction can be divided into Two Phases: Locking (Growing) & Unlocking (Shrinking)
 - **Locking (Growing) Phase:**
 - A transaction applies locks (read or write) on desired data items one at a time.
 - acquires all locks but cannot release any locks.
 - **Unlocking (Shrinking) Phase:**
 - A transaction unlocks its locked data items one at a time.
 - Releases locks but cannot acquire any new locks.
- **Requirement:**
 - For a transaction these two phases must be mutually exclusively, that is, during locking phase unlocking phase must not start and during unlocking phase locking phase must not begin.



Dealing with Deadlock and Starvation

- **Deadlock**

- It is a state that may result when two or more transaction are each waiting for locks held by the other to be released
- Example :

T1

read_lock (Y);

read_item (Y);

write_lock (X);

T2

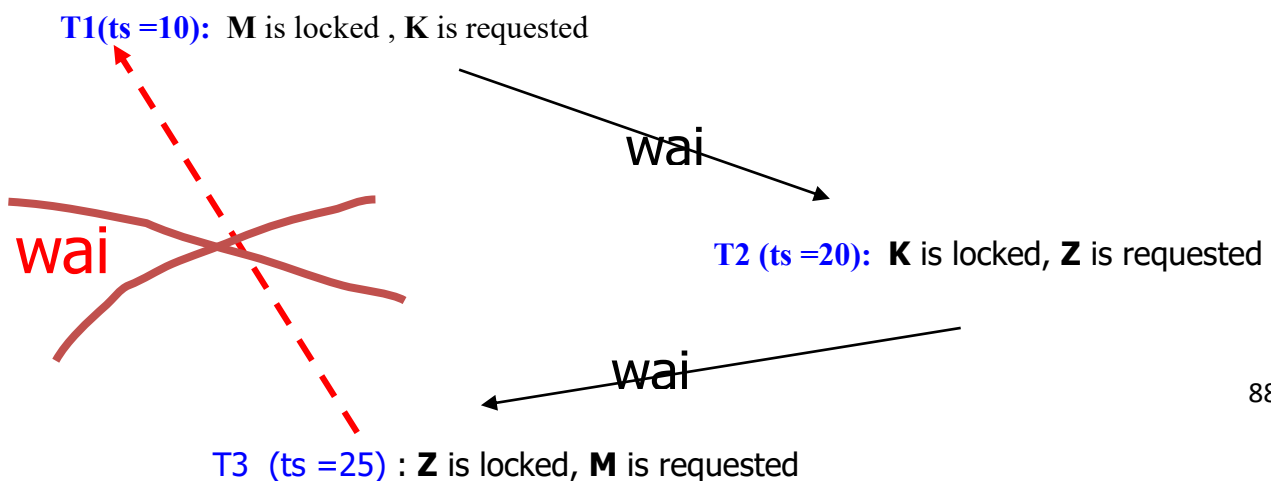
read_lock (X);

read_item (X);

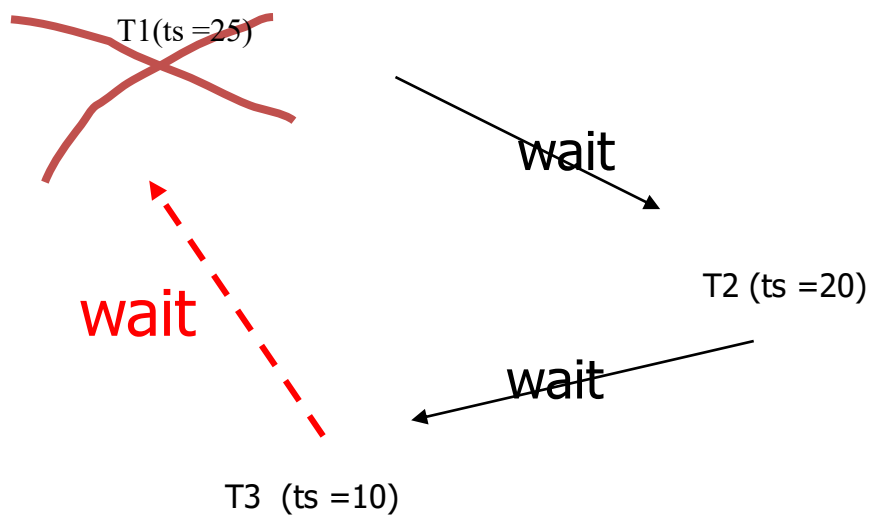
write_lock (Y);

- T1 is in the waiting queue for X which is locked by T2
 - T2 is on the waiting queue for Y which is locked by T1
 - No transaction can continue until the other transaction completes
 - T1 and T2 did follow two-phase policy but they are deadlock
- So the DBMS must either prevent or detect and resolve such deadlock situations
 - **There are possible solutions : Deadlock prevention, deadlock detection and avoidance ,and lock timeouts**
- i. **Deadlock prevention protocol: two possibilities**
- *The conservative two-phase locking*
 - A transaction locks all data items it refers to before it begins execution.

- This way of locking prevents deadlock since a transaction never waits for a data item.
- Limitation : It restrictions concurrency
- **Transaction Timestamp($TS(T)$)**
 - We can prevent deadlocks by giving each transaction a priority and ensuring that lower priority transactions are not allowed to wait for higher priority transactions (or vice versa).
 - One way to assign priorities is to give each transaction a **timestamp** when it starts up.
 - it is a unique identifier given to each transaction based on time in which it is started. i.e if T_1 starts before T_2 , $TS(T_1) < TS(T_2)$
 - The lower the timestamp, the higher the transaction's priority, that is, the oldest transaction has the highest priority.
 - **If a transaction T_i requests a lock and transaction T_j holds a conflicting lock, the lock manager can use one of the following two policies:** Wait-die & Wound-wait
- Wait-die
 - If T_i has higher priority, it is allowed to wait; otherwise it is aborted.
 - An older transaction is allowed to wait on a younger transaction.
 - A younger transaction requesting an item held by an older transaction is aborted
 - If $TS(T_i) < TS(T_j)$, then (T_i older than T_j) T_i is allowed to wait.
 - Otherwise (T_i younger than T_j) Abort T_i (T_i dies) and restart it later with the same timestamp



- Wound-wait
 - The opposite of wait-die
 - If T_i has higher priority, abort T_j ; otherwise T_i waits.
 - A younger transaction is allowed to wait on an older one
 - An older transaction requesting an item held by a younger transaction preempts the younger transaction by aborting it.
 - If $TS(T_i) < TS(T_j)$, then (T_i older than T_j), Abort T_j (T_i wounds T_j) and restart T_j later with the same timestamp
 - Otherwise (T_i younger than T_j) T_i is allowed to wait



ii. Deadlock Detection and resolution

- In this approach, deadlocks are allowed to happen
- The scheduler maintains a wait-for-graph for detecting cycle.
- When a chain like: T_i waits for T_j waits for T_k waits for T_i or T_j occurs, then this creates a cycle.

- When the system is in the state of deadlock , some of the transaction should be aborted by selected (victim) and rolled-back
- This can be done by aborting those transaction: that have made the least work, the one with the lowest locks, and that have the least # of abortion and so on

iii. **Timeouts**

- It uses the period of time that several transaction have been waiting to lock items
- It has lower overhead cost and it is simple
- If the transaction wait for a longer time than the predefined time out period, the system assume that may be deadlocked and aborted it

▪ **Starvation**

- Starvation occurs when a particular transaction consistently waits or restarted and never gets a chance to proceed further while other transaction continue normally
- This may occur , if the waiting method for item locking:
 - Gave priority for some transaction over others
 - Problem in Victim selection algorithm- it is possible that the same transaction may consistently be selected as victim and rolled-back .example In Wound-Wait
- Solution
 - FIFO
 - Allow for transaction that wait for a longer time
 - Give higher priority for transaction that have been aborted for many time

4.2.3 **Timestamp based concurrency control algorithm**

• **Timestamp**

- In lock based concurrency control , conflicting actions of different transactions are ordered by the order in which locks are obtained.
- But here, Timestamp values are assigned based on time in which the transaction are submitted to the system using the current date & time of the system

- A monotonically increasing variable (integer) indicating the age of an operation or a transaction.
 - A larger timestamp value indicates a more recent event or operation.
 - Timestamp based algorithm uses timestamp to serialize the execution of concurrent transactions.
 - **It doesn't use lock, thus deadlock cannot be occurred**
 - In the timestamp ordering, conflicting operation in the schedule shouldn't violate serializable ordering
 - This can be achieved by associating timestamp value (TS) to each database item which is denoted as follow:
- a) Read_TS(x): the read timestamp of x – this is the largest time among all the time stamps of transaction that have successfully read item X
 - b) Write_TS(X): the largest of all the timestamps of transaction that have successfully written item X
- The concurrency control algorithm check whether conflict operation violate the timestamp ordering in the following manner: three options
- i. Basic Timestamp Ordering
 - ❑ Transaction T issues a write_item(X) operation:
 - If $\text{read_TS}(X) > \text{TS}(T)$ or if $\text{write_TS}(X) > \text{TS}(T)$, then a younger transaction has already read/write the values of the data item x before T had a chance to write X . so abort and roll-back T and restarted with a new, larger timestamp. Why is with new timestamp?, is there a difference b/n this timestamp protocol and the 2PL for dead lock prevention?
 - If the condition in part (a) does not exist, then execute write_item(X) of T and set write_TS(X) to TS(T).
 - ❑ Transaction T issues a read_item(X) operation:
 - If $\text{write_TS}(X) > \text{TS}(T)$, then an younger transaction has already written to the data item,so abort and roll-back T and reject the operation.

- If $\text{write_TS}(X) \leq \text{TS}(T)$, then execute $\text{read_item}(X)$ of T and set $\text{read_TS}(X)$ to the larger of $\text{TS}(T)$ and the current $\text{read_TS}(X)$

❑ Limitation: **cyclic restart/starvation** may occur when a transaction is continuously aborted and restarted

ii. **Strict Timestamp Ordering**

1. Transaction T issues a $\text{write_item}(X)$ operation:

- If $\text{TS}(T) > \text{read_TS}(X)$, then delay T until the transaction T' that wrote or read X has terminated (committed or aborted).

2. Transaction T issues a $\text{read_item}(X)$ operation:

- If $\text{TS}(T) > \text{write_TS}(X)$, then delay T until the transaction T' that wrote X has terminated (committed or aborted).

iii. **Thomas's Write Rule : Modification of Basic Time ordering Algorithm**

- **A transaction T issue a $\text{write_item}(X)$ Operation:**
- If $\text{read_TS}(X) > \text{TS}(T)$ then abort and roll-back T and reject the operation.
- If $\text{write_TS}(X) > \text{TS}(T)$, then just ignore the write operation and continue execution. This is because the most recent writes counts in case of two consecutive writes. Example
- If the conditions given in 1 and 2 above do not occur, then execute $\text{write_item}(X)$ of T and set $\text{write_TS}(X)$ to $\text{TS}(T)$.

4.2.4 Multiversion Concurrency Control Techniques

- This approach maintains a number of versions of a data item and allocates the right version to a read operation of a transaction.
- Thus unlike other mechanisms a read operation in this mechanism is never rejected.
- This algorithm uses the concept of view serializability than conflict serializability
- Side effect:
 - Significantly more storage (RAM and disk) is required to maintain multiple versions. To check unlimited growth of versions, a garbage collection is run when some criteria is satisfied.

- Two schemes : based on time stamped ordering & 2PL
- Multiversion technique based on timestamp ordering
 - Assume $X_1, X_2, \dots, X_n$ are the version of a data item X created by a write operation of transactions. With each X_i a $read_TS$ (read timestamp) and a $write_TS$ (write timestamp) are associated.)
 - **read_TS(X_i)**: The read timestamp of X_i is the largest of all the timestamps of transactions that have successfully read version X_i .
 - **write_TS(X_i)**: The write timestamp of X_i is the timestamp of transaction that wrote the value of version X_i .
 - A new version of X_i is created only by a write operation.
- **To ensure serializability, the following two rules are used:**
 - If transaction T issues $write_item(X)$ and version i of X has the highest $write_TS(X_i)$ of all versions of X that is also less than or equal to $TS(T)$, and $read_TS(X_i) > TS(T)$, then abort and roll-back T ; otherwise create a new version X_i and $read_TS(X) = write_TS(X_i) = TS(T)$.
 - If transaction T issues $read_item(X)$, find the version i of X that has the highest $write_TS(X_i)$ of all versions of X that is also less than or equal to $TS(T)$, then return the value of X_i to T , and set the value of $read_TS(X_i)$ to the largest of $TS(T)$ and the current $read_TS(X_i)$.
- Note that: ***Rule two indicates that read request will never be rejected***

ii. Multiversion Two-Phase Locking Using Certify Lock

- Allow a transaction T' to read a data item X while it is write locked by a conflicting transaction T .
- This is accomplished by maintaining two versions of each data item X where one version must always have been written by some committed transaction.
- This means a write operation always creates a new version of X .

Steps

1. X is the committed version of a data item.
2. T creates a second version X' after obtaining a write lock on X .

3. Other transactions continue to read X.
4. T is ready to commit so it obtains a certify lock on X'.
5. The committed version X becomes X'.
6. T releases its certify lock on X', which is X now.

Note:

- In multiversion 2PL read and write operations from conflicting transactions can be processed concurrently.
- This improves concurrency but it may delay transaction commit because of obtaining certify locks on all its writes.
- It avoids cascading abort but like strict two phase locking scheme conflicting transactions may get deadlocked.

4.2.5 Validation (Optimistic) Concurrency Control Schemes

- This technique allow transaction to proceed asynchronously and only at the time of commit, serializability is checked &
- transactions are aborted in case of non-serializable schedules.
- Good if there is little interference among transaction
- It has three phases: **Read, Validation , and Write phase**

i.. Read phase:

- A transaction can read values of committed data items. However, updates are applied only to local copies (versions) of the data items (in database cache).

ii. Validation phase:.

- If the transaction T_i decides that it wants to commit, the DBMS checks whether the transaction could possibly have conflicted with any other concurrently executing transaction.
- While one transaction T_i is being validated , no other transaction can be allowed to commit
- This phase for T_i checks that, for each transaction T_j that is either committed or is in its validation phase, one of the following conditions holds:
 - T_j completes its write phase before T_i starts its read phase.

- T_i starts its write phase after T_j completes its write phase and the read set of T_i has no item in common with the write set of T_j
- Both the read_set and write_set of T_i have no items in common with the write_set of T_j , and T_j completes its read phase before T_i completes its read phase.
- When validating T_i , the first condition is checked first for each transaction T_j , since (1) is the simplest condition to check. If (1) is false then (2) is checked and if (2) is false then (3) is checked.
- If none of these conditions holds, the validation fails and T_i is aborted.

iii. Write phase:

- On a successful validation, transactions' updates are applied to the database; otherwise, transactions are restarted.

4.2.6 Multiple Granularity Locking

- A lockable unit of data defines its granularity
- Granularity can be coarse (entire database) or it can be fine (an attribute of a relation).
- Example of data item granularity:
 - A field of a database record
 - A database record
 - A disk block/ page
 - An entire file
 - The entire database
- Data item granularity significantly affects concurrency control performance.
- Thus, the degree of concurrency is low for coarse granularity and high for fine granularity.
- Example:
 - A transaction that expects to access most of the pages in a file should probably set a lock on the entire file, rather than locking individual pages or records
 - If a transaction that requires to access relatively few pages of the file, it is better to lock those pages

- Similarly , if a transaction access several records on a page , it should lock the entire page and if it access just a few records , it should lock some those records.
- This example will hold true , if a lock on the node locks that node and implicitly all its descendants
- The set of rules which must be followed for producing serializable schedule are
 - The lock compatibility must adhered to.
 - The root of the tree must be locked first, in any mode.
 - A node N can be locked by a transaction T in S or IX mode only if the parent node is already locked by T in either IS or IX mode.
 - A node N can be locked by T in X, IX, or SIX mode only if the parent of N is already locked by T in either IX or SIX mode.
 - T can lock a node only if it has not unlocked any node (to enforce 2PL policy).
 - T can unlock a node, N, only if none of the children of N are currently locked by T.
- To lock a node in S mode, a transaction must first lock all its ancestors in IS mode. Thus, if a transaction locks a node in S mode, no other transaction can have locked any ancestor in X mode;
- Similarly, if a transaction locks a node in X mode, no other transaction can have locked any ancestor in S or X mode.
- These two cases ensures that no other transaction holds a lock on an ancestor that conflicts with the requested S or X lock on the node.

Chapter 5: Database Recovery Techniques

5.1 Database Recovery

i. Purpose of Database Recovery

- To bring the database into the last consistent state, which existed prior to the failure.
- To preserve transaction properties (Atomicity & Durability).
 - The recovery manager of a DBMS is responsible to ensure
 - atomicity by undoing the action of transaction that do not commit
 - Durability by making sure that all actions of committed transaction survive system crash

ii. Types of Failure

- The database may become unavailable for use due to :
 - **Transaction failure:** Transactions may fail because of incorrect input, deadlock.
 - **System failure:** System may fail because of addressing error, application error, operating system fault, RAM failure, etc.
 - **Media failure:** Disk head crash, power disruption, etc.

5.2 Transaction and Recovery

Why recovery is needed:

Whenever a transaction is submitted to the DBMS for execution, the system is responsible for making sure that either all operations in the transaction to be completed successfully or the transaction has no effect on the database or any other transaction.

The DBMS may permit some operations of a transaction T to be applied to the database but a transaction may fail after executing some of its operations

- **What causes a Transaction to fail**

1. A computer failure (system crash):

A hardware or software error occurs in the computer system during transaction execution. If the hardware crashes, the contents of the computer's internal memory may be lost.

2. A transaction or system error:

Some operation in the transaction may cause it to fail, such as integer overflow or division by zero. Transaction failure may also occur because of erroneous parameter values or because of a logical programming error. In addition, *the user may interrupt the transaction during its execution.*

3. Exception conditions detected by the transaction:

- Certain conditions force cancellation of the transaction.
- For example, data for the transaction may not be found. Such as insufficient account balance in a banking database, may cause a transaction, such as a fund withdrawal from that account, to be canceled.

4. Concurrency control enforcement:

The concurrency control method may decide to abort the transaction, to be restarted later, because it violates serializability or because several transactions are in a state of deadlock (see Chapter 2).

5. Disk failure:

Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash. This may happen during a read or a write operation of the transaction.

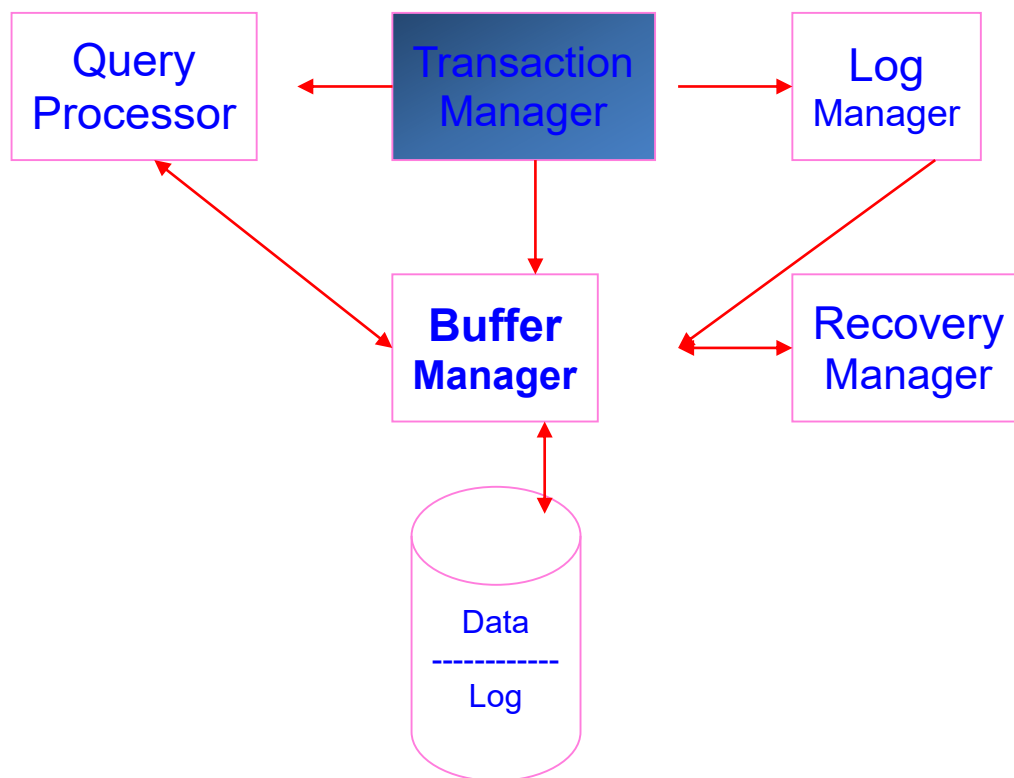
6. Physical problems and catastrophes:

This refers to an endless list of problems that includes power or air-conditioning failure, fire, theft, overwriting disks or tapes by mistake

Transaction Manager : Accepts transaction commands from an application, which tell the transaction manager when transactions begin and end, as well as information about the expectations of the application.

The transaction processor performs the following tasks:

- ✓ Logging: In order to assure **durability**, every change in the database is logged separately on disk.
- ✓ Log manager initially writes the log in buffers and negotiates with the buffer manager to make sure that buffers are written to disk at appropriate times.
- ✓ Recovery Manager: will be able to examine the log of changes and restore the database to some consistent state.



Recovery techniques and facilities

Recovery Algorithms

- Recovery algorithms are techniques to ensure database consistency and transaction atomicity and durability despite failures
 - Focus of this chapter
- Recovery algorithms have two parts

- Actions taken during normal transaction processing to ensure enough information exists to recover from failures
- Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability

Storage Structure

- **Volatile storage:**
 - does not survive system crashes
 - examples: main memory, cache memory
- **Nonvolatile storage:**
 - survives system crashes
 - examples: disk, tape, flash memory, non-volatile (battery backed up) RAM
- **Stable storage:**
 - a mythical form of storage that survives all failures
 - approximated by maintaining multiple copies on distinct nonvolatile media

Stable-Storage Implementation

- Maintain multiple copies of each block on separate disks
 - copies can be at remote sites to protect against disasters such as fire or flooding.
- Failure during data transfer can still result in inconsistent copies: Block transfer can result in
 - Successful completion
 - Partial failure: destination block has incorrect information
 - Total failure: destination block was never updated
- Protecting storage media from failure during data transfer (one solution):
 - Execute output operation as follows (assuming two copies of each block):

1. Write the information onto the first physical block.
 2. When the first write successfully completes, write the same information onto the second physical block.
 3. The output is completed only after the second write successfully completes.
- Protecting storage media from failure during data transfer (cont.):
 - Copies of a block may differ due to failure during output operation. To recover from failure:
 - First find inconsistent blocks:
 1. *Expensive solution*: Compare the two copies of every disk block.
 2. *Better solution*:
 - Record in-progress disk writes on non-volatile storage (Non-volatile RAM or special area of disk).
 - Use this information during recovery to find blocks that may be inconsistent, and only compare copies of these.
 - Used in hardware RAID systems
 - If either copy of an inconsistent block is detected to have an error (bad checksum), overwrite it by the other copy. If both have no error, but are different, overwrite the second block by the first block.

Data Access

- **Physical blocks** are those blocks residing on the disk.
- **Buffer blocks** are the blocks residing temporarily in main memory.
- Block movements between disk and main memory are initiated through the following two operations:
 - **input(B)** transfers the physical block B to main memory.
 - **output(B)** transfers the buffer block B to the disk, and replaces the appropriate physical block there.
- Each transaction T_i has its private work-area in which local copies of all data items accessed and updated by it are kept.

- T_i 's local copy of a data item X is called x_i .
- We assume, for simplicity, that each data item fits in, and is stored inside, a single block.
- Transaction transfers data items between system buffer blocks and its private work-area using the following operations :
 - **read**(X) assigns the value of data item X to the local variable x_i .
 - **write**(X) assigns the value of local variable x_i to data item $\{X\}$ in the buffer block.
 - both these commands may necessitate the issue of an **input**(B_X) instruction before the assignment, if the block B_X in which X resides is not already in memory.
- Transactions
 - Perform **read**(X) while accessing X for the first time;
 - All subsequent accesses are to the local copy.
 - After last access, transaction executes **write**(X).
- **output**(B_X) need not immediately follow **write**(X). System can perform the **output** operation when it deems fit.

Recovery and Atomicity

- Modifying the database without ensuring that the transaction will commit may leave the database in an inconsistent state.
- Consider transaction T_i that transfers \$50 from account A to account B ; goal is either to perform all database modifications made by T_i or none at all.
- Several output operations may be required for T_i (to output A and B). A failure may occur after one of these modifications have been made but before all of them are made.
- To ensure atomicity despite failures, we first output information describing the modifications to stable storage without modifying the database itself.
- We study two approaches:
 - **log-based recovery**, and
 - **shadow-paging**
- We assume (initially) that transactions run serially, that is, one after the other.

System log

- To recover from system failure , the system keeps information about the change in the system log
- Strategy for recovery may be summarized as :
 - Recovery from catastrophic
 - If there is extensive damage to the wide portion of the database
 - This method restore a past copy of the database from the backup storage and reconstructs operation of a committed transaction from the back up log up to the time of failure
 - Recovery from non-catastrophic failure
 - When the database is not physically damaged but has become inconsistent
 - The strategy uses *undoing and redoing* some operations in order to restore to a consistent state : *example*
- For instance,
 - If failure occurs between commit and database buffers being flushed to secondary storage then, to ensure durability, recovery manager has to *redo (roll forward)* transaction's updates.
 - If transaction had not committed at failure time, recovery manager has to *undo (rollback)* any effects of that transaction for atomicity. Example,

Transaction Log

- For recovery from any type of failure data values prior to modification (BFIM - BeFore Image) and the new value after modification (AFIM – AFTer Image) are required.
- These values and other information is stored in a sequential file (appended file) called *Transaction log*
- These log files become very useful in bringing back the system to a stable state after a system crash.
- A sample log is given below. Back P and Next P point to the previous and next log records of the same transaction.

Data Caching

- Data items to be modified are first stored into database cache by the Cache Manager (CM) and after modification they are flushed (written) to the disk
- When DBMS request for read /write operation on some item
 - It check the requested data item is in the cache or not
 - If it is not, the appropriate disk block are copied to the cache
 - If the cache is already full, some buffer replacement policy can be used .
Like
 - Least recent used (LRU)
 - FIFO
 - While replacing buffers , first of all the updated value on that buffer should be saved on the appropriate block in the data base

Write-Ahead Logging

- When ***in-place*** update (immediate or deferred) is used then log is necessary for recovery
- This log must be available to recovery manager
- This is achieved by *Write-Ahead Logging (WAL)* protocol. WAL states that
 - **For Undo:** Before a data item's AFIM is flushed to the database disk (overwriting the BFIM) its BFIM must be written to the log and the log must be saved on a stable store (log disk).
 - **For Redo:** Before a transaction executes its commit operation, all its AFIMs must be written to the log and the log must be saved on a stable store.

☐ Standard Recovery Terminology

- Possible ways for flushing database cache to database disk:
 - i. No-Steal: Cache cannot be flushed before transaction commit.
 - ii. Steal: Cache can be flushed before transaction commits.

Advantage:

- it avoids the need for a very large buffer space to store all updated pages in the memory
- ii. Force: if all Cache updates are immediately flushed (forced) to disk when a transaction commits-----→ force writing
- iii. No-Force: if Cached are flushed to a disk when the need arise after a committed transaction

Advantage:

- an updated pages of a committed transaction may still be in the buffer when an other transaction needs to update
- If this page is updated by multiple transaction, it eliminates the I/O cost to read that page again ,
- These give rise to four different ways for handling recovery:
 - Steal/No-Force (Undo/Redo)
 - Steal/Force (Undo/No-redo)
 - No-Steal/No-Force (No-undo/Redo)
 - No-Steal/Force (No-undo/No-redo)

Check pointing

- Log file is used to recover failed DB but we may not know how far back in the log to search. Thus
- Checkpoint is a Point of synchronization between database and log file.
- Time to time (randomly or under some criteria) the database flushes its buffer to database disk to minimize the task of recovery.
- When failure occurs, redo all transactions that committed since the checkpoint and undo all transactions active at time of crash.
- In previous example ,Figure 2, on slide18 , with checkpoint at time t_c , changes made by T2 and T3 have been written to secondary storage. Thus: *only redo T4 and T5, undo transactions T1 and T6.*
- At what interval to take a check pointing may be measured :
 - In terms of time , say m minutes after the last check point

- The # t of committed transaction since the last check point
- The following steps defines a checkpoint operation:
 - Suspend execution of transactions temporarily.
 - Force write modified buffer data to disk.
 - Write a [checkpoint] record to the log, save the log to disk.
 - Resume normal transaction execution.

Transaction **Roll-back (Undo)** and **Roll-Forward (Redo)**

- If transaction fails for what so ever reason after updating the data base it must be necessary to roll back
- If a transaction T is roll back , any transaction that has read the value of some data item X written by T must be also be roll back
- Similarly once S is roll back, any transaction R that has read the values of some data item Y written by S must aslo be roll back, and so on..
- To maintain such cases, a transaction's operations are redone or undone.
 - **Undo:** Restore all BFIMs on to disk (Remove all AFIMs).
 - **Redo:** Restore all AFIMs on to disk.
- Database recovery is achieved either by performing only Undos or only Redos or by a combination of the two. These operations are recorded in the log as they happen.

Data Update and Recovery Scheme : Three types

- i. Deferred Update (No Undo/Redo)
 - The data update goes as follows:
 - A set of transactions records their updates in the log.
 - At commit point under WAL scheme these updates are saved on database disk.
 - No undo is required because no AFIM is flushed to the disk before a transaction commits.

- After reboot from a failure the log is used to redo all the transactions affected by this failure.
- Limitation: *out of buffer space may be happened because transaction changes must be held in the cache buffer until the commit point*
- Type of deferred updated recovery environment
 - Single User and Multiple user environment

ii. Deferred Update in a single-user system

- There is no concurrent data sharing in a single user system.
- The data update goes as follows:
 - A set of transactions records their updates in the log.
 - At commit point under WAL scheme these updates are saved on database disk.

b) Deferred Update with concurrent users

- This environment requires some concurrency control mechanism to guarantee **isolation** property of transactions.
- In a system recovery, transactions which were recorded in the log after the last checkpoint were **redone**.
- The recovery manager may scan some of the transactions recorded before the checkpoint to get the AFIMs.
- Two tables are required for implementing this protocol:
 - **Active table:** All active transactions are entered in this table.
 - **Commit table:** Transactions to be committed are entered in this table.
- During recovery, all transactions of the **commit** table are redone and all transactions of **active** tables are ignored since none of their AFIMs reached the database.
- It is possible that a **commit** table transaction may be **redone** twice but this does not create any inconsistency because of a redone is “**idempotent**”,
 - that is, one redone for an AFIM is equivalent to multiple redone for the same AFIM.

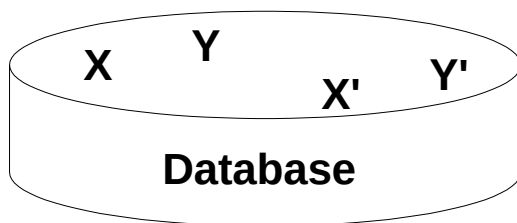
ii. Recovery Techniques Based on Immediate Update

▪ Undo/No-redo Algorithm

- In this algorithm AFIMs of a transaction are flushed to the database disk under WAL before it commits.
- For this reason the recovery manager **undoes** all transactions during recovery.
- No transaction is **redone**.
- It is possible that a transaction might have completed execution and ready to commit but this transaction is also **undone**.

iii. Shadow Paging

- Maintain two page tables during life of a transaction: *current* page and *shadow* page table.
- When transaction starts, two pages are the same.
- Shadow page table is never changed thereafter and is used to restore database in event of failure.
- During transaction, current page table records all updates to database.
- When transaction completes, current page table becomes shadow page table.



X and Y: Shadow copies of data items

X' and Y': Current copies of data items

The ARIES Recovery Algorithm

- It is designed to work with a steal , no- force approach
- It is based on: three principles
 - **WAL** (Write Ahead Logging)

- Any change to the database object is first recorded in the log
- The record on the log must first be save on the disk and then
- The database object is written to disk
- **Repeating history during redo:**
 - ARIES will retrace all actions of the database system prior to the crash to reconstruct the database state when the crash occurred.
 - Transaction that were un committed at the time of crash are undone
- **Logging changes during undo:**
 - It will prevent ARIES from repeating the completed undo operations if a failure occurs during recovery, which causes a restart of the recovery process.

Data Update : Four types

- **Deferred Update:**
 - All transaction updates are recorded in the local workspace (cache)
 - All modified data items in the cache is then written after a transaction ends its execution or after a fixed number of transactions have completed their execution.
 - During commit the updates are first recorded on the log and then on the database
 - If a transaction fails before reaching its commit point undo is not needed because it didn't change the database yet
 - If a transaction fails after commit (writing on the log) but before finishing saving to the data base redoing is needed from the log
- **Immediate Update:**
 - As soon as a data item is modified in cache, the disk copy is updated.
 - These update are first recorded *on the log and on the disk* by force writing, before the database is updated
 - If a transaction fails after recording some change to the database but before reaching its commit point , this will be rolled back

- **Shadow update:**
 - The modified version of a data item does not overwrite its disk copy but is written at a separate disk location.
 - Multiple version of the same data item can be maintained
 - Thus the old value (before image BFIM) and the new value (AFIM) are kept in the disk
 - No need of Log for recovery
- **In-place update:** The disk version of the data item is overwritten by the cache version.
- Information for ARIES to accomplish recovery procedures includes:
 - Log
 - Transaction Table &
 - Dirty page table

i. **The Log and Log Sequence Number (LSN)**

- The log is a history of actions executed by the DBMS in terms of a file of records stored in disk for recovery purpose
- A unique LSN is associated with every log record
 - LSN increases monotonically and indicates the disk address of the log record it is associated with.
 - In addition, each data page stores the LSN of the latest log record corresponding to a change for that page.
- A log record is written for each of the following actions:
 - updating a page
 - i. After modifying the page , an update record is appended to the log tail.
 - ii. The LSN of the page is then set to the LSN of the update log record

- commit- when a transaction decides to commit , it force writes a commit type log record containing the transaction id
- Abort- when a transaction is aborted , an abort type log containing the transaction id is appended to the log
- End-when a transaction is aborted or committed , an end type log record containing transaction id is appended to the log
- undo update- when transaction is roll back its updates are undone
- A log record stores
 - the previous LSN of that transaction
 - the transaction ID
 - the type of log record.
 - For a write operation the following additional information is logged:
 - i. Page ID for the page that includes the item
 - ii. Length of the updated item
 - iii. Its offset from the beginning of the page
 - iv. BFIM of the item
- ii. The **Transaction table** and the **Dirty Page table**
 - For efficient recovery following tables are also stored in the log **during checkpointing**:
 - **Transaction table**: Contains an entry for each active transaction, with information such as transaction ID, transaction status(in progresses, committed or aborted) and the LSN of the most recent log record for the transaction.
 - **Dirty Page table**: Contains an entry for each dirty page in the buffer, which includes the page ID and the LSN corresponding to the earliest update to that page.
- iii. Check pointing
 - A checkpointing does the following:

- Writes a begin_checkpoint record in the log
 - Writes an end_checkpoint record in the log. With this record the contents of transaction table and dirty page table are appended to the end of the log.
 - Writes the LSN of the begin_checkpoint record to a special file. This special file is accessed during recovery to locate the last checkpoint information.
- To reduce the cost of checkpointing and allow the system to continue to execute transactions, ARIES uses “**fuzzy check pointing**”.

(a)

Lsn	Last_lsn	Tran_id	Type	Page_id	Other_information
1	0	T_1	update	C	...
2	0	T_2	update	B	...
3	1	T_1	commit		...
4	begin checkpoint				
5	end checkpoint				
6	0	T_3	update	A	...
7	2	T_2	update	C	...
8	7	T_2	commit		...

(b)

TRANSACTION TABLE			DIRTY PAGE TABLE	
Transaction_id	Last_lsn	Status	Page_id	Lsn
T_1	3	commit	C	1
T_2	2	in progress	B	2

(c)

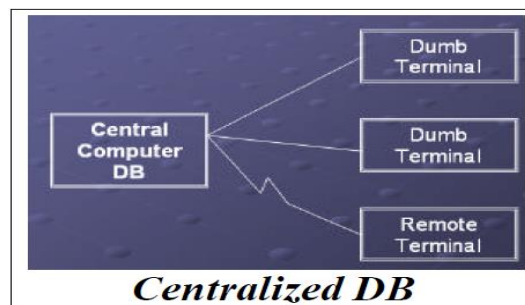
TRANSACTION TABLE			DIRTY PAGE TABLE	
Transaction_id	Last_lsn	Status	Page_id	Lsn
T_1	3	commit	C	1
T_2	8	commit	B	2
T_3	6	in progress	A	6

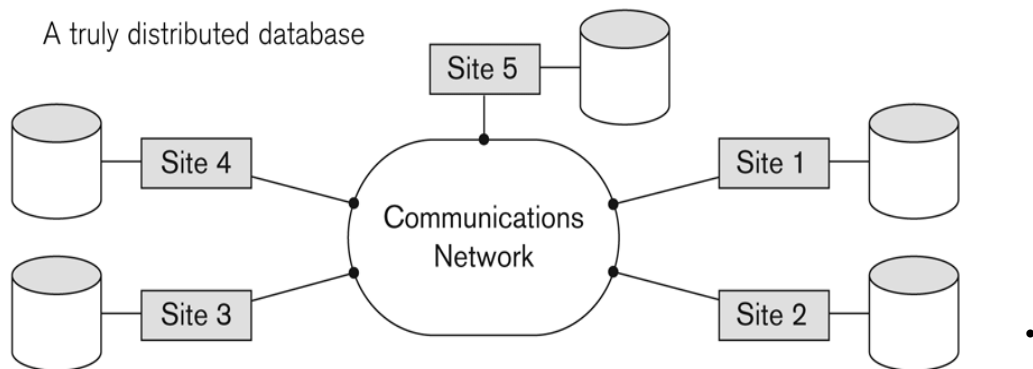
An example of recovery in ARIES. (a) The log at point of crash.
 (b) The Transaction and Dirty Page Tables at time of checkpoint.
 (c) The Transaction and Dirty Page Tables after the analysis phase.

Chapter 6: Distributed Databases and Client-Server Architectures

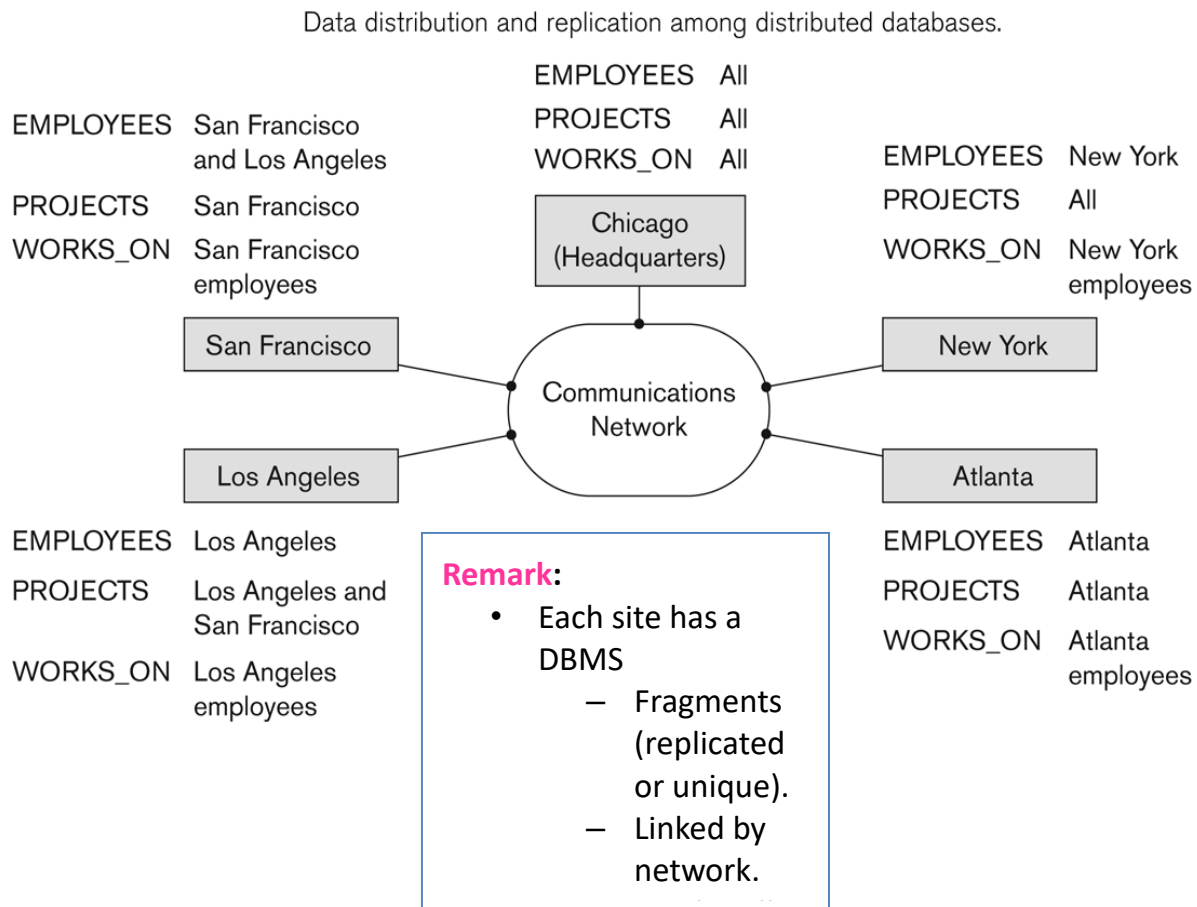
6.1 Distributed Database Concepts

- A transaction can be executed by multiple networked computers in a unified manner.
- A **distributed database (DDB)** processes Unit of execution (a transaction) in a distributed manner.
- **A distributed database (DDB) can be defined as :**
 - A collection of multiple logically related database distributed over a computer network, and a distributed database management system as a software system that manages a distributed database while making the distribution transparent to the user.
 - The physical placement of data (files, relations, etc.) which is not known to the user (distribution transparency).





- The EMPLOYEE, PROJECT, and WORKS_ON tables may be fragmented horizontally and stored with possible replication as shown below.



❑ Advantages of DDB :

i. Distribution and Network transparency:

- Users do not have to worry about operational details of the network.
 - » There is Location transparency, which refers to freedom of issuing command from any location without affecting its working.
 - » Then there is Naming transparency, which allows access to any names object (files, relations, etc.) from any location.

ii. Replication transparency:

- It allows to store copies of a data at multiple sites as shown in the above diagram.
- This is done to minimize access time to the required data.

iii. Fragmentation transparency:

- Allows to fragment a relation horizontally (create a subset of rows of a relation) or vertically (create a subset of columns of a relation).

iv. Increased reliability and availability:

- ❑ Reliability refers to system live time, that is, system is running efficiently most of the time. Availability is the probability that the system is continuously available (usable or accessible) during a time interval.
- ❑ A distributed database system has multiple nodes (computers) and if one fails then others are available to do the job.

v. Improved performance:

- ❑ A distributed DBMS fragments the database to keep data closer to where it is needed most.
- ❑ This reduces data management (access and modification) time significantly.

vi. **Easier expansion (scalability):**

- ❑ Allows new nodes (computers) to be added anytime without changing the entire configuration.

Disadvantages of Distributed Database

- i. Complexity- The data replication , failure recovery , network management ...make the system more complex than the central DBMSs
- ii. Cost- Since DDBMS needs more people and more hardware, maintaining and running the system can be more expensive than the centralized system .
- iii. Problem of connecting Dissimilar Machine- Additional layers of operation system software are needed to translate and coordinate the flow of data between machines.
- iv. Data integrity and security problem - Because data maintained by distributed systems can be accessed at locations in the network, controlling the integrity of a database can be difficult.

6.2 **Data Replication and Fragmentation: Distributed data storage**

- There are two approaches to store the relation in the distributed database : **Replication and Fragmentation**

I. **Data Replication**

- The system maintain several identical copies of the relation & store each copy at a different site
- In general it enhance the performance of read operation and increase the availability of data to read only transaction. However, update transactions incur greater overhead

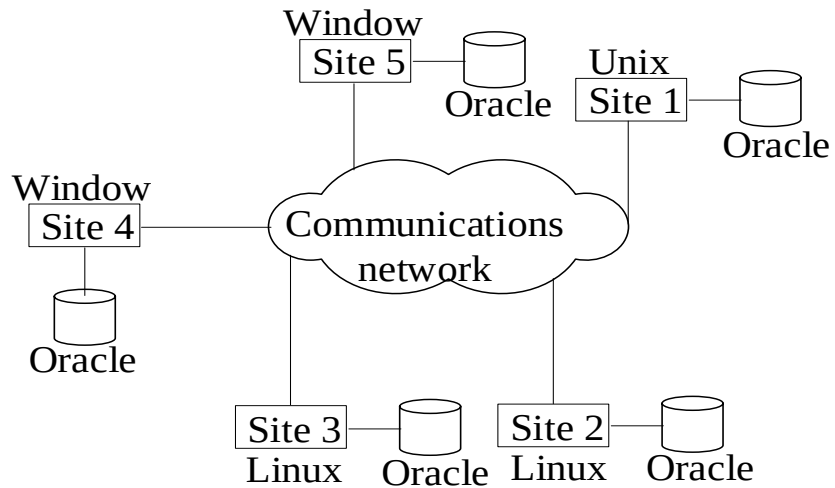
II. **Data Fragmentation**

- Split a relation into logically related and correct parts.
- The main reasons for fragmenting a relation are
 - Efficiency- data that is not needed by the local applications is not stored
 - Parallelism – a transaction can be divided into several subqueries that operate on fragments which will increase the degree of concurrency
- but reconstruction of the whole relation will require accessing data from all sites containing part of the relation

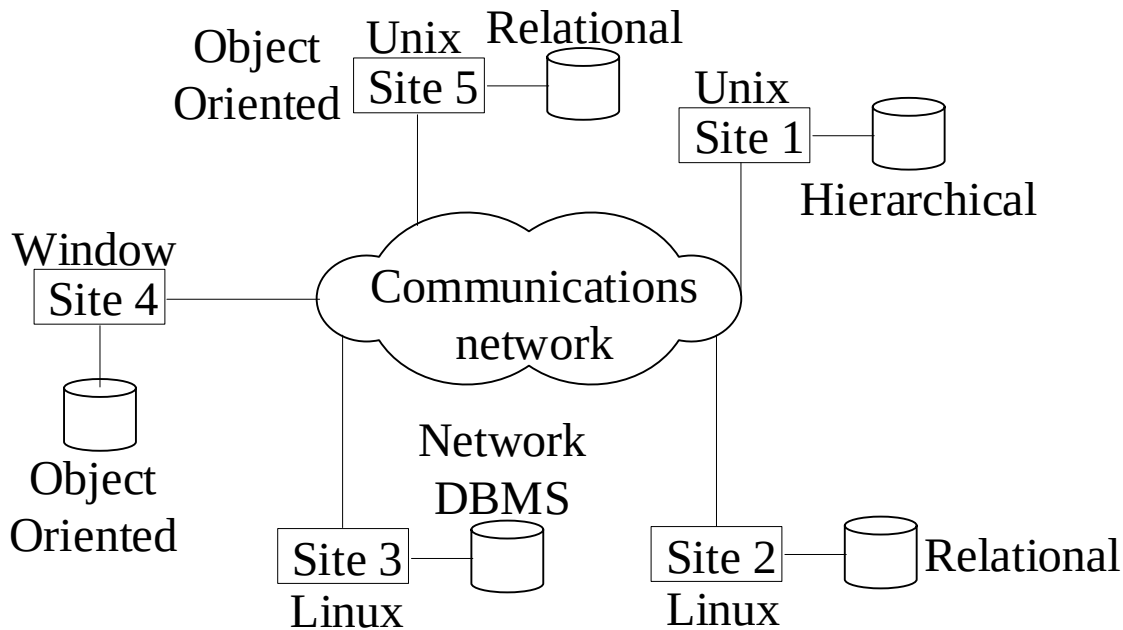
- A relation can be fragmented in two ways:
 - **Horizontal fragmentation**
 - It is a horizontal subset of a relation which contain those of rows which satisfy selection conditions.
 - Consider the Employee relation with selection condition ($DNO = 5$). All rows satisfy this condition will create a subset which will be a horizontal fragment of Employee relation.
 - A selection condition may be composed of several conditions connected by AND or OR.
 - **Vertical fragmentation**
 - It is a subset of a relation which is created by a subset of columns. Thus a vertical fragment of a relation will contain values of selected columns.
 - Consider the Employee relation. A vertical fragment of can be created by keeping the values of Name, Bdate, Sex, and Address.
 - Because there is no condition for creating a vertical fragment, each fragment must include the primary key attribute of the parent relation Employee. In this way all vertical fragments of a relation are connected.
- **Representation**
 - There three rules that must be followed during fragmentation
 - **Completeness** – if a relation r is decomposed into fragments $r_1, r_2 \dots r_n$, each data item that can be found in r must appear in at least one fragment
 - **Reconstruction** – it must be possible to define a relation operation that will reconstruct the relation r from fragments
 - **Disjointness** –if a data item d_i appears in fragment r_i , then it shouldn't appear in any other fragment

6.3 Types of Distributed Database Systems

- Homogeneous
 - All sites of the database system have identical setup, i.e., same database system software.
 - The system may have little or no local autonomy
 - The underlying operating systems can be a mixture of Linux, Window, Unix, etc.



- Heterogeneous
 - At least one of the database must be from different vendor : two variants
 - Federated: Each site may run different database system but the data access is managed through a single conceptual schema.
 - This implies that the degree of local autonomy is minimum. Each site must adhere to a centralized access policy. There may be a global schema.
 - Multidatabase: There is no one conceptual global schema. For data access a schema is constructed dynamically as needed by the application software.



6.4 Query Processing in Distributed Databases

▪ Issues

- Cost of transferring data (files and results) over the network.
 - This cost is usually high, so some optimization is necessary.
 - **Example:** suppose there are three sites. Where the relation Employee at site 1, Department at Site 2 and no relation at site 3
 - Employee at site 1. 10,000 rows. Row size = 100 bytes. Table size = 10^6 bytes.
 - Department at Site 2. 100 rows. Row size = 35 bytes. Table size = 3,500 bytes.
- And a query is initiated from S3 to retrieve employees [First Name (15 byte long), Last name (15 byte long) and Department name (10 byte long) total of 40 bytes]
 - **Q: For each employee, retrieve employee Fname, Lname, and department name**
 - Q: $\delta_{Fname, Lname, Dname}$ (Employee $Dno = Dnumber$ Department)

▪ Assumption

- The result of this query will have 10,000 rows, assuming that every employee is related to a department.
- Suppose each result row 40 bytes long. The query is submitted at site 3 and the result is sent to this site.
- Problem: Employee and Department relations are not present at site 3.
- what is your best strategy that can optimize data transportation cost?
- Strategies : Minimizing data transfer.
 - Transfer Employee and Department to site 3.
 - Total transfer bytes = 1,000,000 + 3500 = 1,003,500 bytes.
 - Transfer Employee to site 2, execute join at site 2 and send the result to site 3.
 - Transferring employees data from site 1 to site 2: 1,000,000 bytes
 - Query result size = 40 * 10,000 = 400,000 bytes.
 - Total transfer size = 1,000,000 + 400,000 = 1,400,000 bytes.
 - Transfer Department relation to site 1, execute the join at site 1, and send the result to site 3.
 - Data Transfer from site 2 to site 1: 3500 bytes
 - Query result size = 40 * 10,000 = 400,000 bytes
 - Total bytes transferred = 3500 + 400,000 = 403,500 bytes.
- Preferred approach: strategy 3.

Example 2 : Consider the query

- Q': For each department, retrieve the department name ,Fname and LName of the department manager
- Relational Algebra expression:
 - $\delta_{Fname, Lname, Dname} (Employee \bowtie_{Mgrssn = SSN} Department)$
- The result of this query will have 100 tuples, assuming that every department has a manager, the execution strategies are:

- Transfer Employee and Department to the result site and perform the join at site 3.
 - Total bytes transferred = $1,000,000 + 3500 = 1,003,500$ bytes.
- Transfer Employee to site 2, execute join at site 2 and send the result to site 3.
 - Site 1--→ Site 2: 1,000,000
 - Site2--→ site3: Query result size = $40 * 100 = 4000$ bytes.
 - Total transfer size = $4000 + 1,000,000 = 1,004,000$ bytes.
- Transfer Department relation to site 1, execute join at site 1 and send the result to site 3.
 - Total transfer size = $4000 + 3500 = 7500$ bytes.

Preferred strategy: Choose strategy 3.

Example 3: Now suppose the result is needed at site2. Possible strategies :

1. Transfer Employee relation to site 2, execute the query and present the result to the user at site 2.
 - Total transfer size = 1,000,000 bytes .
2. Transfer Department relation to site 1, execute join at site 1 and send the result back to site 2.
 - Total transfer size
 - $Q = 400,000 + 3500 = 403,500$ bytes

Preferred strategy: Choose strategy 2.

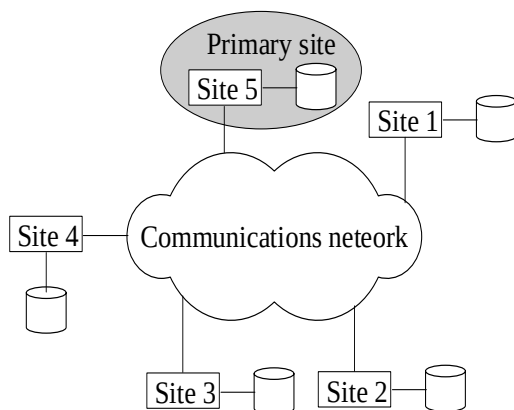
6.5 Concurrency Control and Recovery

- Distributed Databases encounter a number of concurrency control and recovery problems which are not present in centralized databases. Some of them are listed below.
 - **Dealing with multiple copies of data items:**
 - The concurrency control must maintain global consistency. Likewise the recovery mechanism must recover all copies and maintain consistency after recovery.
 - **Failure of individual sites:**

- Database availability must not be affected due to the failure of one or two sites and the recovery scheme must recover them before they are available for use.
- Communication link failure:
 - This failure may create network partition which would affect database availability even though all database sites may be running.
- Distributed commit:
 - A transaction may be fragmented and they may be executed by a number of sites. This requires a two or three-phase commit approach for transaction commit.
- Distributed deadlock:
 - Since transactions are processed at multiple sites, two or more sites may get involved in deadlock. This must be resolved in a distributed manner.

6.5.1 Distributed Concurrency control

- i. *Primary site technique:* A single site is designated as a primary site which serves as a coordinator for transaction management.



- Transaction management:
 - Concurrency control and commit are managed by this site.
 - In two phase locking, this site manages locking and releasing data items. If all transactions follow two-phase policy at all sites, then serializability is guaranteed.
 - Advantages:

- An extension to the centralized two phase locking so implementation and management is simple.
 - Data items are locked only at one site but they can be accessed at any site.
 - Disadvantages:
 - All transaction management activities go to primary site which is likely to overload the site.
 - If the primary site fails, the entire system is inaccessible.
 - To aid recovery a backup site is designated which behaves as a shadow of primary site. In case of primary site failure, backup site can act as primary site.
- ii. *Primary Copy Technique:*
- In this approach, instead of a site, a data item partition is designated as primary copy. To lock a data item just the primary copy of the data item is locked.
 - Advantages:
 - Since primary copies are distributed at various sites, a single site is not overloaded with locking and unlocking requests.
 - Disadvantages:
 - Identification of a primary copy is complex. A distributed directory must be maintained, possibly at all sites.

Recovery from a coordinator failure

- In both approaches a coordinator site or copy may become unavailable. This will require the selection of a new coordinator.
 - Primary site approach with no backup site:
 - Aborts and restarts all active transactions at all sites. Elects a new coordinator and initiates transaction processing.
 - Primary site approach with backup site:
 - Suspends all active transactions, designates the backup site as the primary site and identifies a new back up site.
 - Primary site receives all transaction management information to resume processing.

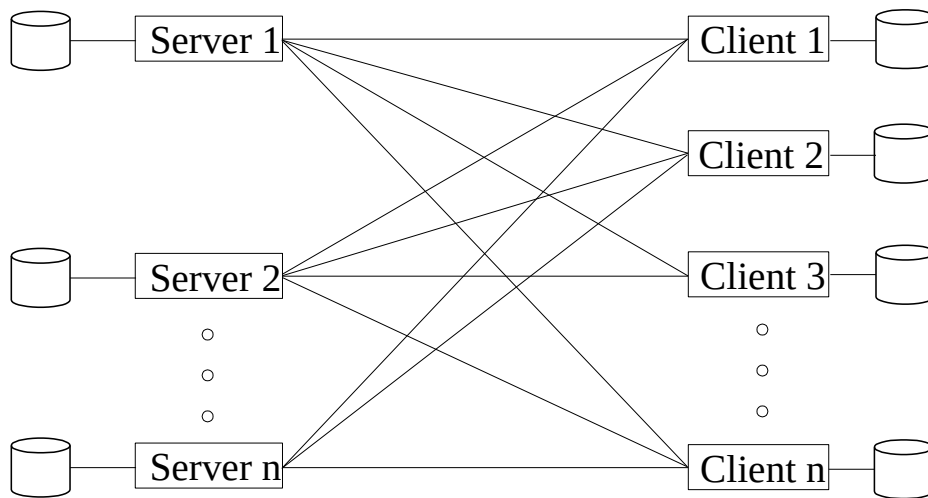
- Primary and backup sites fail or no backup site:
 - Use election process to select a new coordinator site.

iii. **Concurrency control based on voting:**

- There is no primary copy of coordinator.
- Send lock request to sites that have data item.
- If majority of sites grant lock then the requesting transaction gets the data item.
- Locking information (grant or denied) is sent to all these sites.
- To avoid unacceptably long wait, a time-out period is defined. If the requesting transaction does not get any vote information then the transaction is aborted.

6.6 Client-Server Database Architecture

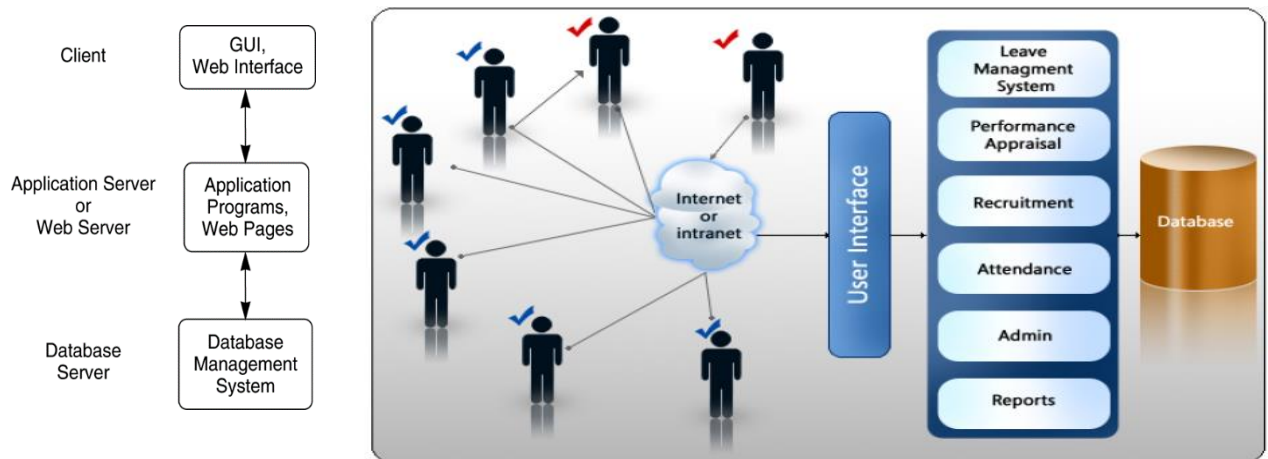
- It consists of clients running client software, a set of servers which provide all database functionalities and a reliable communication infrastructure.



three-tier client/server architecture.

- Many Web applications use an architecture called the three-tier architecture, which adds an intermediate layer between the client and the database server. This intermediate layer called the Web server. This server plays an intermediary role by storing business rules(constraints) that are used to access data from the database server.

- It can also *improve database security* by checking a client's credentials before forwarding a request to the database server. The intermediate server accepts requests from the client, processes the request and sends database commands to the database server, and then *acts as a conduit* for passing (partially) processed data from the database server to the clients



- Clients reach server for desired service, but server does reach clients.
- The server software is responsible for local data management at a site, much like centralized DBMS software.
- The client software is responsible for most of the distribution function.
- The communication software manages communication among clients and servers.
- The processing of a SQL queries goes as follows:
 - Client parses a user query and decomposes it into a number of independent subqueries. Each subquery is sent to appropriate site for execution.
 - Each server processes its query and sends the result to the client.
 - The client combines the results of subqueries and produces the final result.